

Deep Learning Basics

Interview-focused notes for ML placements, AI engineer roles, research internships, and projects.

Artificial Neuron

1. Overview

An artificial neuron is the smallest computational unit in a neural network. It takes input features, computes a weighted sum plus bias, applies an activation function, and produces an output. It is used in MLPs, CNNs, RNNs, Transformers, recommendation models, and nearly every deep learning system.

2. Intuition

Think of a neuron as a scoring rule. For house-price prediction, it may give positive weight to area, negative weight to distance from city center, add a bias, then pass the score through a function to produce a useful output.

3. Prerequisites

Linear algebra, dot product, vectors, scalar functions, basic calculus, gradients, supervised learning, Python/NumPy.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Inputs	Feature vector $\mathbf{x}$	Carries data	Pixels, tabular features	Shape conventions
Weights	Learnable vector $\mathbf{w}$	Controls feature importance	High weight for useful feature	Parameter learning
Bias	Learnable scalar b	Shifts decision boundary	Allows non-zero output when input is zero	Why bias is needed
Activation	Nonlinear function f	Enables nonlinear modeling	ReLU, sigmoid, tanh	Why linear stacks collapse

5. Algorithm / Working Process

Input vector $\mathbf{x}$ is multiplied by weights $\mathbf{w}$, bias b is added, activation f is applied, and output $\hat{y}$ is returned. During training, the output is compared with target y , loss is computed, and gradients update $\mathbf{w}$ and b .

6. Mathematical Foundation

Weighted pre-activation:

$$z = w^T x + b$$

$$a = f(z)$$

For a batch X :

$$Z = XW + b$$

$$A = f(Z)$$

Learning updates:

$$w := w - \eta * dL/dw$$

$$b := b - \eta * dL/db$$

7. Practical Implementation

```
import torch
import torch.nn as nn

class ArtificialNeuron(nn.Module):
    def __init__(self, in_features):
        super().__init__()
        self.linear = nn.Linear(in_features, 1)

    def forward(self, x):
        return torch.relu(self.linear(x))

x = torch.tensor([[1.0, 2.0, 3.0]])
neuron = ArtificialNeuron(3)
print(neuron(x))
```

8. Code Explanation

`nn.Linear` stores weights and bias. `forward` computes $wx + b$ and applies ReLU. PyTorch autograd records operations so gradients can be computed during backpropagation.

9. Training / Evaluation

Use a dataset with inputs and labels, split into train/validation/test, choose a loss, and optimize parameters. A single neuron can solve only simple linear problems unless paired with nonlinear features.

10. Complexity and Cost

For d input features, one neuron has $d + 1$ parameters. Forward cost is $O(d)$ per sample. Memory cost is tiny.

11. Common Use Cases

Linear scoring, logistic regression, hidden units in neural networks, feature detectors in CNNs, attention projection components.

12. Common Mistakes

Forgetting bias, using no activation in hidden layers, wrong input shape, assuming one neuron can model complex nonlinear data, using sigmoid everywhere and causing saturation.

13. Edge Cases / Limitations

A single neuron has limited expressiveness. With linear activation, it is only a linear model. With hard threshold activation, it is not differentiable.

14. Variations

Linear neuron for regression, sigmoid neuron for binary probability, ReLU neuron for hidden layers, softmax output neuron group for multiclass classification.

15. Related Topics

Perceptron is a threshold neuron. Logistic regression is a sigmoid neuron trained with cross entropy. MLPs combine many neurons into layers.

16. Interview Questions

1. What is an artificial neuron?
A learnable unit computing $f(\mathbf{w}^T \mathbf{x} + \mathbf{b})$.
2. Why do we need bias?
To shift the function/decision boundary.
3. What happens without activation?
Stacked layers remain a single linear transformation.
4. What are trainable parameters?
Weights and bias.
5. How are parameters learned?
By minimizing loss using gradient descent variants.
6. What is pre-activation?
 $\mathbf{z} = \mathbf{w}^T \mathbf{x} + \mathbf{b}$ before activation.
7. What is activation output?
 $\mathbf{a} = f(\mathbf{z})$.
8. Can one neuron solve XOR?
No, XOR is not linearly separable.
9. What controls neuron capacity?
Input dimension, activation, and composition with other neurons.
10. Where is the neuron used in Transformers?
Linear projections and feed-forward layers.

17. Practice Tasks

Implement one neuron in NumPy, train it on linearly separable data, compare sigmoid vs ReLU output, debug shape mismatch, extend to multiple outputs.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Single-Neuron Classifier	Binary classification from scratch	NumPy	Synthetic blobs	Shows fundamentals
Feature Weight Explorer	Visualizes learned weights	Python, matplotlib	Iris binary subset	Interpretability
Neuron Playground	Interactive activation demo	Streamlit	Generated data	Teaching/demo value

19. Quick Revision

Key idea: learn a weighted score and activate it. Main formula: $a=f(w^T x+b)$. Use inside all neural networks. Trap: no nonlinearity means no deep nonlinear learning. Interview one-liner: a neuron is a differentiable learnable feature detector.

20. Final Cheat Sheet

Definition: basic neural unit. Input/output: vector to scalar or vector. Steps: weighted sum, bias, activation. Hyperparameters: activation, input size. Pros: simple and composable. Cons: weak alone. Best use: building block of deep models.

Perceptron

1. Overview

The perceptron is an early binary classifier that uses a linear decision boundary and a step function. It is useful historically and conceptually because it explains linear separability, weight updates, and why multilayer networks became necessary.

2. Intuition

It is like a yes/no rule: if weighted evidence crosses a threshold, predict class **1**; otherwise predict **0** or **-1**.

3. Prerequisites

Vectors, dot product, binary classification, linearly separable data, learning rate, basic Python.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Linear boundary	Hyperplane separates classes	Defines model capacity	$w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$	Linear separability
Step activation	Hard class decision	Non-probabilistic output	1 if $z \geq 0$ else 0	Not differentiable
Update rule	Correct mistakes only	Simple online learning	Move boundary toward correct class	Convergence theorem
Separability	Perfect linear split exists	Required for convergence	AND is separable, XOR is not	Classic question

5. Algorithm / Working Process

Initialize weights and bias. For each training example, compute prediction. If prediction is wrong, update weights toward the true label. Repeat for epochs until no mistakes or max epochs.

6. Mathematical Foundation

For labels $y \in \{-1, +1\}$:

```

y_hat = sign(w^T x + b)
if y * (w^T x + b) <= 0:
    w := w + eta * y * x
    b := b + eta * y

```

The perceptron converges in finite updates if data is linearly separable.

7. Practical Implementation

```

import numpy as np

class Perceptron:
    def __init__(self, lr=0.1, epochs=20):
        self.lr = lr
        self.epochs = epochs

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1])
        self.b = 0.0
        for _ in range(self.epochs):
            mistakes = 0
            for xi, yi in zip(X, y):
                if yi * (np.dot(self.w, xi) + self.b) <= 0:
                    self.w += self.lr * yi * xi
                    self.b += self.lr * yi
                    mistakes += 1
            if mistakes == 0:

```

```

        break

    def predict(self, X):
        return np.where(X @ self.w + self.b >= 0, 1, -1)

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([-1, -1, -1, 1]) # AND
model = Perceptron()
model.fit(X, y)
print(model.predict(X))

```

8. Code Explanation

Weights start at zero. Each misclassified sample changes the boundary. The `predict` method applies the sign rule. The AND dataset is linearly separable, so the perceptron can learn it.

9. Training / Evaluation

Use accuracy for balanced binary data. Standardize features if scales differ. The perceptron may never converge on non-separable data, so use max epochs.

10. Complexity and Cost

Training cost is $O(\text{epochs} * n * d)$. Inference cost is $O(d)$ per sample. Memory is $O(d)$.

11. Common Use Cases

Educational classifier, online linear classification, basis for understanding neural networks and support vector machines.

12. Common Mistakes

Using labels $\{0, 1\}$ with the $\{-1, +1\}$ update, expecting probabilities, using it on XOR, forgetting feature scaling, no stopping condition.

13. Edge Cases / Limitations

Fails on non-linearly separable data, sensitive to feature scaling and sample order, does not produce calibrated probabilities.

14. Variations

Averaged perceptron improves stability. Kernel perceptron handles nonlinear boundaries. Logistic regression replaces step activation with sigmoid and optimizes cross entropy.

15. Related Topics

Artificial neuron generalizes perceptron with differentiable activations. SVM also learns a linear separator but maximizes margin. MLP solves XOR using hidden layers.

16. Interview Questions

1. What is a perceptron?
A linear binary classifier with a threshold decision.
2. What is the update rule?
 $w = w + \text{eta} * y * x$ for misclassified samples.
3. When does it converge?
When data is linearly separable.
4. Can it solve XOR?
No.
5. Why is step activation problematic for deep learning?
It is not differentiable.
6. Difference from logistic regression?
Logistic regression outputs probabilities and uses cross entropy.
7. What is a decision boundary?
The hyperplane $w^T x + b = 0$.
8. Why use bias?
To shift the boundary.
9. What is online learning?
Updating after each example.
10. What if data is not separable?
Perceptron can keep making mistakes.

17. Practice Tasks

Train on AND/OR/XOR, plot decision boundary, convert labels correctly, compare perceptron with logistic regression, implement averaged perceptron.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Linear Gate Learner	Learns Boolean gates	NumPy	AND/OR/XOR	Clear fundamentals
Online Spam Toy	Updates per email	Python	SMS spam subset	Online learning
Boundary Visualizer	Animates updates	Streamlit	Synthetic 2D data	Interview demo

19. Quick Revision

Key idea: linear classifier corrected on mistakes. Formula: $\text{sign}(w^T x + b)$. Use for separable binary tasks. Trap: cannot solve XOR. Interview one-liner: perceptron is the simplest trainable linear neuron.

20. Final Cheat Sheet

Definition: binary linear classifier. Input/output: features to class. Steps: score, threshold, update if wrong. Hyperparameters: learning rate, epochs. Metric: accuracy. Pros: simple. Cons: linear only. Best use: learning fundamentals.

Multilayer Perceptron

1. Overview

A multilayer perceptron (MLP) is a feed-forward neural network made of fully connected layers and nonlinear activations. It is used for tabular data, embeddings, classification heads, regression, and the feed-forward blocks inside Transformers.

2. Intuition

An MLP stacks many neurons so early layers learn simple patterns and later layers combine them into more complex patterns, like combining edges into shapes and shapes into objects.

3. Prerequisites

Artificial neurons, matrix multiplication, activation functions, loss functions, gradient descent, backpropagation, PyTorch basics.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Dense layer	Every input connects to every output	General feature mixing	<code>nn.Linear(784,128)</code>	Parameter count
Hidden layer	Intermediate representation	Enables hierarchy	ReLU layer	Depth vs width
Nonlinearity	Activation between linear layers	Prevents collapse to linear model	ReLU	Why activation is needed
Output head	Maps features to task output	Task-specific	logits for classes	Softmax vs logits

5. Algorithm / Working Process

Input passes through linear layer, activation, optional normalization/dropout, repeated hidden layers, and final output layer. Training uses forward pass, loss computation, backpropagation, and optimizer update. Inference uses only forward pass, usually with dropout disabled.

6. Mathematical Foundation

For layer `l`:

```
z_l = a_{l-1} W_l + b_l
a_l = f_l(z_l)
```

For classification:

```
logits = a_L
P(y=k|x) = softmax(logits)_k
L = -log P(y=true|x)
```

7. Practical Implementation

```
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, in_dim, hidden_dim, num_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, num_classes),
        )

    def forward(self, x):
        return self.net(x)

model = MLP(in_dim=784, hidden_dim=128, num_classes=10)
x = torch.randn(32, 784)
logits = model(x)
print(logits.shape)
```

8. Code Explanation

The model maps 784-dimensional flattened images to 10 class logits. Hidden layers learn nonlinear representations. The final layer returns raw logits, which should be passed to `nn.CrossEntropyLoss`.

9. Training / Evaluation

Prepare normalized tensors, split train/validation/test, use cross entropy for classification or MSE for regression, monitor accuracy/loss, tune learning rate, hidden size, number of layers, dropout, and weight decay.

10. Complexity and Cost

A dense layer has `input_dim * output_dim + output_dim` parameters. MLPs can become expensive for high-dimensional inputs, which is why CNNs are preferred for images.

11. Common Use Cases

Tabular prediction, classification heads, regression, recommendation systems, feature transformation, Transformer feed-forward networks.

12. Common Mistakes

Applying softmax before `CrossEntropyLoss`, no normalization of inputs, too large MLP for small data, forgetting `model.eval()`, overfitting without validation.

13. Edge Cases / Limitations

Poor spatial inductive bias for images, parameter-heavy, may overfit small datasets, struggles with sequences unless features are engineered.

14. Variations

Deep MLP, residual MLP, MLP-Mixer, gated MLP, dropout MLP, batch-normalized MLP.

15. Related Topics

Perceptron is one layer with threshold. CNNs add spatial structure. Transformers use MLP blocks after attention. Residual connections help train deep MLPs.

16. Interview Questions

1. What is an MLP?
A stack of fully connected layers with nonlinear activations.
2. Why hidden layers?
They learn intermediate representations.
3. Why activation functions?
Without them, the network is linear.
4. What are logits?
Raw unnormalized class scores.
5. How many parameters in `Linear(m,n)`?
 $m \cdot n + n$.
6. Why can MLP overfit?
Many parameters and weak inductive bias.
7. MLP vs CNN?
CNN shares weights and uses locality.
8. MLP for tabular data?
Often useful after preprocessing but tree models can be strong.
9. How train an MLP?
Forward, loss, backward, optimizer step.
10. What improves MLP generalization?
Dropout, weight decay, normalization, more data.

17. Practice Tasks

Build MNIST MLP, tune hidden width, compare ReLU/tanh, add dropout, plot train vs validation loss.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
MNIST MLP	Digit classifier	PyTorch	MNIST	Baseline DL project
Tabular Risk Model	Predicts churn/default	PyTorch, sklearn	Telco churn	Applied ML
Embedding Classifier	Classifies text embeddings	PyTorch	AG News embeddings	Modern AI workflow

19. Quick Revision

Key idea: dense nonlinear function approximator. Formula: $a_l = f(a_{l-1}W_l + b_l)$. Use for tabular, heads, embeddings. Trap: softmax before cross entropy. Interview one-liner: MLPs compose linear maps and nonlinearities to learn complex functions.

20. Final Cheat Sheet

Definition: feed-forward dense network. Input/output: vector to prediction. Steps: dense, activation, repeat, output. Hyperparameters: depth, width, activation, LR, dropout. Metrics: accuracy, F1, RMSE. Pros: flexible. Cons: data hungry and parameter-heavy. Best use: vector features.

Forward Propagation

1. Overview

Forward propagation is the process of passing inputs through a neural network to compute predictions and loss. It is used during both training and inference.

2. Intuition

It is the network's "calculation phase": data flows from input to output, layer by layer, like a pipeline transforming raw features into decisions.

3. Prerequisites

Matrix multiplication, layers, activations, model parameters, batch dimension, loss functions.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Batch input	Multiple samples processed together	Efficient GPU use	(32, 784)	Shape tracking

Subtopic	Meaning	Why it matters	Example	Interview angle
Layer transform	Applies learnable operation	Builds representation	Linear, Conv	Parameter sharing
Activation	Nonlinear transform	Adds expressiveness	ReLU	Linear collapse
Loss computation	Compares prediction and target	Training objective	CE, MSE	Logits vs probabilities

5. Algorithm / Working Process

Load a batch, pass it through each layer in order, compute logits or predictions, compute loss if labels are available, return outputs for evaluation or backpropagation.

6. Mathematical Foundation

```

a_0 = x
z_1 = a_{l-1} W_1 + b_1
a_1 = f_1(z_1)
y_hat = g(a_L)
L = loss(y_hat, y)

```

7. Practical Implementation

```

import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3),
)

x = torch.randn(5, 4)
y = torch.tensor([0, 1, 2, 1, 0])
logits = model(x)
loss = nn.CrossEntropyLoss()(logits, y)
print(logits.shape, loss.item())

```

8. Code Explanation

The batch has 5 samples and 4 features. The model outputs 3 logits per sample. `CrossEntropyLoss` combines log-softmax and negative log likelihood.

9. Training / Evaluation

During training, forward propagation stores intermediate values for gradients. During evaluation, use `torch.no_grad()` to save memory and `model.eval()` to change dropout/batchnorm behavior.

10. Complexity and Cost

Cost is the sum of all layer operations. Dense layer cost is $O(\text{batch} * \text{in_dim} * \text{out_dim})$. Forward pass is usually cheaper than full training because training also needs backward pass and optimizer updates.

11. Common Use Cases

Prediction, feature extraction, loss computation, validation, deployment inference.

12. Common Mistakes

Wrong tensor shape, applying softmax before CE loss, forgetting `no_grad` in evaluation, using training mode during inference, not moving data/model to same device.

13. Edge Cases / Limitations

Large batches may exceed GPU memory. Dynamic control flow can make debugging shapes harder. BatchNorm behavior differs between train and eval.

14. Variations

Static graph forward pass, dynamic graph forward pass, autoregressive forward pass, teacher-forced forward pass, streaming inference.

15. Related Topics

Backpropagation uses values from forward propagation. Loss functions define what forward outputs optimize. Mixed precision changes forward data types.

16. Interview Questions

1. What is forward propagation?
Computing outputs from inputs through model layers.
2. Is it used in inference?
Yes.
3. Why keep intermediate activations during training?
Backpropagation needs them.
4. What is a batch dimension?
Number of samples processed together.
5. What are logits?
Raw scores before probability normalization.
6. How does eval mode affect forward pass?
Disables dropout and uses BatchNorm running stats.
7. Why use `torch.no_grad()`?
It avoids gradient graph storage.

8. What causes shape errors?
Mismatched dimensions between layers.
9. What is output of classification forward pass?
Usually logits.
10. Forward vs backward cost?
Backward is often about 2x forward cost.

17. Practice Tasks

Trace shapes through an MLP, write a manual NumPy forward pass, compare train/eval outputs with dropout, profile batch sizes, debug a dimension mismatch.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Shape Tracer	Logs layer shapes	PyTorch hooks	Any model	Debugging skill
Inference API	Serves forward pass	FastAPI, PyTorch	MNIST	Deployment basics
Feature Extractor	Saves embeddings	PyTorch	CIFAR/ImageNet subset	Transfer learning

19. Quick Revision

Key idea: input to prediction. Formula: $a_l = f(a_{l-1}W_l + b_l)$. Use in training and inference. Trap: training/eval mode mismatch. Interview one-liner: forward propagation is the deterministic computation graph evaluation.

20. Final Cheat Sheet

Definition: pass data through model. Input/output: batch to predictions/loss. Steps: layers, activations, output, loss. Hyperparameters: batch size. Metrics: task-specific. Pros: efficient on GPU. Cons: memory-heavy during training. Best use: prediction and loss computation.

Backpropagation

1. Overview

Backpropagation computes gradients of the loss with respect to model parameters using the chain rule. It is the core training algorithm behind modern deep learning.

2. Intuition

After a wrong prediction, backpropagation asks: "Which weights contributed to this error, and by how much?" It sends blame backward through the network.

3. Prerequisites

Calculus, partial derivatives, chain rule, matrix multiplication, loss functions, forward propagation, gradient descent.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Gradient	Direction of steepest loss increase	Guides updates	dL/dw	Sign of update
Chain rule	Derivative of composition	Enables deep gradients	$dL/dw = dL/da * da/dz * dz/dw$	Core derivation
Computational graph	Operations and dependencies	Autograd representation	PyTorch graph	Dynamic vs static
Gradient accumulation	Gradients add by default	Supports large effective batch	<code>zero_grad()</code> needed	Common bug

5. Algorithm / Working Process

Run forward pass, compute loss, start from $dL/dL=1$, propagate gradients backward layer by layer using local derivatives, store gradients in parameters, optimizer updates parameters.

6. Mathematical Foundation

For $z = wx + b$, $a=f(z)$, loss L :

$$\begin{aligned} dL/dw &= dL/da * da/dz * dz/dw \\ dL/db &= dL/da * da/dz \\ dz/dw &= x \end{aligned}$$

Gradient descent:

$$\theta := \theta - \eta * \text{gradient}_{\theta} L$$

7. Practical Implementation

```
import torch
import torch.nn as nn

model = nn.Linear(2, 1)
x = torch.tensor([[1.0, 2.0]])
```

```
y = torch.tensor([[1.0]])

criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

pred = model(x)
loss = criterion(pred, y)

optimizer.zero_grad()
loss.backward()
optimizer.step()

print(loss.item())
```

8. Code Explanation

`loss.backward()` computes gradients using autograd. `zero_grad()` clears old gradients. `step()` updates weights using the optimizer.

9. Training / Evaluation

Backpropagation is used only during training. Evaluation should use `torch.no_grad()`. Monitor gradient norms to detect vanishing or exploding gradients.

10. Complexity and Cost

Backward pass usually costs about 1-2 times the forward pass and stores activations, increasing memory use.

11. Common Use Cases

Training neural networks, fine-tuning LLMs, optimizing embeddings, differentiable simulation, neural style transfer.

12. Common Mistakes

Forgetting `zero_grad`, calling `.detach()` accidentally, using non-differentiable operations, in-place ops breaking gradients, exploding gradients without clipping.

13. Edge Cases / Limitations

Very deep networks can suffer vanishing/exploding gradients. Discrete decisions are not directly differentiable. Long sequences can be memory expensive.

14. Variations

Automatic differentiation, manual backprop, truncated BPTT, gradient checkpointing, higher-order gradients.

15. Related Topics

Forward propagation creates the graph. Optimizers use gradients. Residual connections improve gradient flow. Mixed precision uses gradient scaling.

16. Interview Questions

1. What is backpropagation?
Chain-rule gradient computation through a network.
2. Why is chain rule important?
Networks are composed functions.
3. What does `loss.backward()` do?
Computes parameter gradients.
4. Why call `zero_grad()`?
PyTorch accumulates gradients by default.
5. What is gradient descent?
Updating parameters opposite the gradient.
6. What causes vanishing gradients?
Repeated multiplication by small derivatives.
7. What causes exploding gradients?
Repeated multiplication by large derivatives.
8. How fix exploding gradients?
Gradient clipping, normalization, initialization.
9. Does inference need backprop?
No.
10. What is autograd?
Automatic differentiation engine.

17. Practice Tasks

Manually derive gradients for one neuron, implement backprop in NumPy, compare manual and PyTorch gradients, inspect gradient norms, debug missing gradients.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Manual Backprop MLP	Trains from scratch	NumPy	XOR	Strong fundamentals
Gradient Monitor	Tracks layer gradient norms	PyTorch	MNIST	Debugging skill
Differentiable Fitter	Fits curve parameters	PyTorch	Synthetic	Optimization insight

19. Quick Revision

Key idea: compute gradients backward. Formula: chain rule. Use for training. Trap: accumulated gradients. Interview one-liner: backprop efficiently computes all parameter gradients in a computational graph.

20. Final Cheat Sheet

Definition: gradient computation algorithm. Input/output: loss to gradients. Steps: forward, loss, backward, update. Hyperparameters: learning rate via optimizer. Pros: efficient. Cons: memory-heavy and gradient instability. Best use: neural network training.

Activation Functions

1. Overview

Activation functions introduce nonlinearity into neural networks. Without them, any stack of linear layers equals one linear layer.

2. Intuition

Activations decide how strongly a neuron should fire. They bend the model's function so it can fit curves, boundaries, and complex patterns.

3. Prerequisites

Functions, derivatives, neural layers, gradients, optimization.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Nonlinearity	Breaks linear composition	Enables complex functions	ReLU	Universal approximation
Differentiability	Allows gradient learning	Backprop needs derivatives	sigmoid derivative	Dead/saturated gradients
Range	Output interval	Affects stability	sigmoid (0,1)	Output layer choice
Smoothness	Derivative behavior	Affects optimization	GELU vs ReLU	Modern architectures

5. Algorithm / Working Process

For each layer, compute pre-activation z , apply activation $a=f(z)$, pass a to the next layer. During backpropagation, multiply by $f'(z)$.

6. Mathematical Foundation

$$z = Wx + b$$
$$a = f(z)$$
$$dL/dz = dL/da * f'(z)$$

Common activations: ReLU, sigmoid, tanh, GELU, SiLU, Leaky ReLU, softmax.

7. Practical Implementation

```
import torch
import torch.nn as nn

x = torch.linspace(-3, 3, steps=7)
activations = {
    "relu": nn.ReLU(),
    "sigmoid": nn.Sigmoid(),
    "tanh": nn.Tanh(),
    "gelu": nn.GELU(),
}

for name, fn in activations.items():
    print(name, fn(x))
```

8. Code Explanation

The code applies different activations to the same input range, showing differences in output range and saturation.

9. Training / Evaluation

Choose hidden activations based on gradient behavior and architecture. ReLU/GELU are common in deep networks. Sigmoid/softmax are common for output probabilities.

10. Complexity and Cost

Elementwise activations are usually cheap. Softmax is more expensive because it normalizes across a dimension.

11. Common Use Cases

Hidden nonlinearities, binary outputs, multiclass outputs, gates in LSTMs, Transformer feed-forward layers.

12. Common Mistakes

Using sigmoid in deep hidden layers, applying softmax before `CrossEntropyLoss`, ignoring dead ReLUs, wrong output activation for loss.

13. Edge Cases / Limitations

Saturating activations can cause vanishing gradients. ReLU can die for negative inputs. Softmax can overflow without numerical stabilization.

14. Variations

Leaky ReLU, ELU, GELU, SiLU/Swish, Mish, PReLU, hard sigmoid.

15. Related Topics

ReLU improves gradient flow. Sigmoid supports binary probability. Tanh is zero-centered. Softmax normalizes multiclass logits.

16. Interview Questions

1. Why use activation functions?
To add nonlinearity.
2. What if all activations are removed?
The network becomes linear.
3. Which activation is common in hidden layers?
ReLU/GELU.
4. Which activation for binary output?
Sigmoid, or logits with BCEWithLogitsLoss.
5. Which activation for multiclass output?
Softmax, or logits with CrossEntropyLoss.
6. What is saturation?
Regions where derivative is near zero.
7. What is dead ReLU?
A ReLU neuron always outputting zero.
8. Why is tanh often better than sigmoid in hidden layers?
It is zero-centered.
9. Why is GELU used in Transformers?
Smooth gating-like behavior.
10. Are activations trainable?
Usually no, except variants like PReLU.

17. Practice Tasks

Plot activation curves, train MLP with ReLU vs tanh, inspect gradients, implement numerically stable softmax, compare GELU and ReLU.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Activation Benchmark	Compares convergence	PyTorch	MNIST	Practical tuning
Activation Visualizer	Shows curves/gradients	Streamlit	Synthetic	Teaching artifact
Dead ReLU Detector	Finds inactive neurons	PyTorch hooks	CIFAR subset	Debugging skill

19. Quick Revision

Key idea: nonlinear layer output. Formula: $\mathbf{a} = f(\mathbf{z})$. Use hidden and output layers. Trap: wrong activation-loss pairing. Interview one-liner: activations make deep networks more than stacked linear algebra.

20. Final Cheat Sheet

Definition: nonlinear transform. Input/output: tensor to tensor. Steps: apply elementwise or normalize dimension. Hyperparameters: type. Pros: expressiveness. Cons: saturation/dead units. Best use: hidden representations and probability outputs.

ReLU

1. Overview

ReLU, or Rectified Linear Unit, is a widely used hidden-layer activation that outputs zero for negative inputs and the input itself for positive inputs.

2. Intuition

ReLU is like a gate: negative evidence is shut off, positive evidence passes through unchanged.

3. Prerequisites

Activation functions, derivatives, backpropagation, gradient descent.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Piecewise linear	Two simple regions	Easy optimization	$\max(0, x)$	Derivative
Sparsity	Many zeros	Efficient representations	inactive neurons	Regularization effect
Non-saturation positive side	Gradient is 1 for $x > 0$	Helps deep training	CNN hidden layers	Vanishing gradient relief
Dead ReLU	Always outputs zero	Stops learning	large negative bias	How to fix

5. Algorithm / Working Process

For every input value, return x if positive and 0 otherwise. In backward pass, pass gradient through for positive inputs and block it for negative inputs.

6. Mathematical Foundation

$$\begin{aligned}\text{ReLU}(x) &= \max(0, x) \\ \text{ReLU}'(x) &= 1 \text{ if } x > 0, 0 \text{ if } x < 0\end{aligned}$$

At $x=0$, derivative is undefined; frameworks use a subgradient, usually 0 .

7. Practical Implementation

```
import torch
import torch.nn as nn

x = torch.tensor([-2.0, -0.5, 0.0, 1.5])
relu = nn.ReLU()
print(relu(x))
```

8. Code Explanation

Negative inputs become zero, positive inputs stay unchanged. In a neural network, this makes hidden activations sparse and nonlinear.

9. Training / Evaluation

ReLU is usually paired with He initialization. Watch for dead neurons, especially with high learning rates or bad initialization.

10. Complexity and Cost

ReLU is $O(n)$ over tensor elements and extremely cheap. It has no parameters.

11. Common Use Cases

CNNs, MLPs, simple feed-forward networks, baseline deep learning models.

12. Common Mistakes

Using ReLU at output for negative-valued regression, ignoring dead ReLU, using Xavier initialization instead of He for deep ReLU networks.

13. Edge Cases / Limitations

Not zero-centered, dead neurons can occur, unbounded positive output can contribute to exploding activations.

14. Variations

Leaky ReLU allows small negative slope. PReLU learns negative slope. ELU smooths negative side. GELU is smoother and common in Transformers.

15. Related Topics

He initialization is designed for ReLU. BatchNorm can stabilize ReLU inputs. Leaky ReLU fixes dead ReLU partly.

16. Interview Questions

1. Formula for ReLU?
 $\max(0, x)$.
2. Why popular?
Simple, cheap, good gradients for positive inputs.
3. Derivative of ReLU?
1 for positive, 0 for negative.
4. What is dead ReLU?
Neuron stuck outputting zero.
5. How prevent dead ReLU?
Lower LR, better init, Leaky ReLU.
6. Is ReLU differentiable at zero?
No, subgradient is used.
7. ReLU vs sigmoid?
ReLU saturates less on positive side.
8. ReLU for output layer?
Only when output must be non-negative.
9. Best initialization?
He/Kaiming.
10. Is ReLU parameterized?
No.

17. Practice Tasks

Plot ReLU and derivative, count zero activations in a trained model, compare ReLU/Leaky ReLU, test high LR dead neurons, use He initialization.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
ReLU Health Dashboard	Tracks inactive units	PyTorch	MNIST	Debugging
Activation Comparison	Benchmarks activations	PyTorch	Fashion-MNIST	Model tuning
Init Experiment	He vs Xavier	PyTorch	CIFAR subset	Research mindset

19. Quick Revision

Key idea: pass positives, zero negatives. Formula: $\max(0, x)$. Use hidden layers. Trap: dead neurons. Interview one-liner: ReLU made deep networks easier to train by reducing saturation.

20. Final Cheat Sheet

Definition: piecewise linear activation. Input/output: tensor to tensor. Steps: threshold at zero.
Hyperparameters: none. Pros: fast, sparse, good gradients. Cons: dead ReLU. Best use: hidden layers.

Sigmoid

1. Overview

Sigmoid maps any real number to $(0,1)$, making it useful for binary probabilities and gates.

2. Intuition

It squashes a raw score into a probability-like value: very negative means near 0, very positive means near 1.

3. Prerequisites

Exponentials, binary classification, logits, cross entropy, gradients.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Squashing	Maps real to $(0,1)$	Probability output	spam probability	Logit meaning
Saturation	Flat near 0 and 1	Vanishing gradients	<code>sigmoid(10)</code>	Deep hidden issue
Binary output	One probability	Natural for yes/no	fraud/not fraud	BCE loss
Gates	Controls information flow	LSTM/GRU gates	forget gate	Modern use

5. Algorithm / Working Process

Compute a logit z , apply sigmoid, interpret output as probability of positive class. Training usually uses binary cross entropy.

6. Mathematical Foundation

```
sigma(x) = 1 / (1 + e^(-x))
sigma'(x) = sigma(x)(1 - sigma(x))
```

Binary cross entropy:

$$L = -[y \log(p) + (1-y) \log(1-p)]$$

7. Practical Implementation

```
import torch
import torch.nn as nn

logits = torch.tensor([[0.2], [-1.0], [3.0]])
targets = torch.tensor([[1.0], [0.0], [1.0]])

loss = nn.BCEWithLogitsLoss()(logits, targets)
```

```
probs = torch.sigmoid(logits)
print(probs, loss.item())
```

8. Code Explanation

`BCEWithLogitsLoss` expects raw logits and applies a numerically stable sigmoid internally. Use `torch.sigmoid` only for interpretation or inference probabilities.

9. Training / Evaluation

Use binary cross entropy, ROC-AUC, PR-AUC, accuracy, precision, recall, and F1 depending on class imbalance. Tune decision threshold instead of assuming `0.5`.

10. Complexity and Cost

Elementwise $O(n)$. Exponential makes it slightly more expensive than ReLU but still cheap.

11. Common Use Cases

Binary classification, multilabel classification, LSTM gates, attention masks, logistic regression.

12. Common Mistakes

Applying sigmoid before `BCEWithLogitsLoss`, using sigmoid for mutually exclusive multiclass classification, thresholding before loss, ignoring class imbalance.

13. Edge Cases / Limitations

Saturates for large magnitude logits, not zero-centered, can cause vanishing gradients in hidden layers.

14. Variations

Hard sigmoid, temperature-scaled sigmoid, logistic regression, sigmoid gates in recurrent networks.

15. Related Topics

Softmax generalizes probability normalization to multiclass. Tanh is related: $\tanh(x) = 2 * \text{sigmoid}(2x) - 1$. BCE pairs with sigmoid.

16. Interview Questions

1. Formula?
 $1 / (1 + e^{-x})$.
2. Output range?
 $(0, 1)$.
3. Use case?
Binary/multilabel probability.
4. Derivative?
 $\text{sigma}(x)(1 - \text{sigma}(x))$.

5. Why not hidden layers?
Saturation causes vanishing gradients.
6. BCEWithLogitsLoss vs BCELoss?
BCEWithLogitsLoss is numerically stable and takes logits.
7. Sigmoid vs softmax?
Sigmoid independent labels, softmax mutually exclusive labels.
8. What is a logit?
Raw log-odds score.
9. Threshold always 0.5?
No, tune based on cost/metric.
10. What happens for logit 0?
Probability 0.5.

17. Practice Tasks

Implement sigmoid in NumPy, plot saturation, train binary classifier, tune threshold for F1, compare BCE and BCEWithLogitsLoss.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Spam Probability Model	Binary spam detection	PyTorch/sklearn	SMS spam	Practical classification
Multilabel Tagger	Predicts multiple tags	PyTorch	MovieLens tags	Sigmoid vs softmax
Threshold Tuner	Optimizes F1/precision	Python	Imbalanced fraud data	Interview-ready metrics

19. Quick Revision

Key idea: logits to probabilities. Formula: $\sigma(x) = 1/(1+e^{-x})$. Use binary/multilabel output. Trap: double sigmoid with BCEWithLogitsLoss. Interview one-liner: sigmoid converts a score into an independent positive-class probability.

20. Final Cheat Sheet

Definition: logistic activation. Input/output: real to $(0,1)$. Steps: exponentiate and normalize.
Hyperparameters: threshold outside training. Metrics: AUC, F1, precision/recall. Pros: interpretable. Cons: saturation. Best use: binary/multilabel outputs.

Tanh

1. Overview

Tanh maps real values to $(-1, 1)$ and is a zero-centered activation often used in older neural networks and recurrent models.

2. Intuition

Tanh is like sigmoid shifted and stretched so negative evidence becomes negative output and positive evidence becomes positive output.

3. Prerequisites

Activation functions, derivatives, exponentials, gradient descent.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Zero-centered	Output around 0	Easier optimization than sigmoid	hidden state values	Sigmoid vs tanh
Saturation	Flat near -1 and 1	Can vanish gradients	large inputs	Deep network issue
Symmetry	Odd function	Balanced activations	$\tanh(-x) = -\tanh(x)$	Optimization
Recurrent use	Bounded hidden state	Stable state values	vanilla RNN	Why LSTMs use tanh

5. Algorithm / Working Process

Apply tanh elementwise to pre-activation values. It keeps values bounded between -1 and 1 while preserving sign.

6. Mathematical Foundation

$$\begin{aligned}\tanh(x) &= (e^x - e^{-x}) / (e^x + e^{-x}) \\ \tanh'(x) &= 1 - \tanh(x)^2\end{aligned}$$

Relation to sigmoid:

$$\tanh(x) = 2 * \text{sigmoid}(2x) - 1$$

7. Practical Implementation

```
import torch
```

```
x = torch.tensor([-3.0, -1.0, 0.0, 1.0, 3.0])
print(torch.tanh(x))
```

8. Code Explanation

The output is negative for negative input, zero at zero, and positive for positive input. Large magnitudes saturate near -1 or 1 .

9. Training / Evaluation

Works better than sigmoid for hidden layers because it is zero-centered, but ReLU/GELU are more common in deep feed-forward networks.

10. Complexity and Cost

Elementwise $O(n)$. Uses exponentials, so cost is higher than ReLU but usually minor.

11. Common Use Cases

RNN hidden states, LSTM candidate cell values, bounded regression outputs, older MLPs.

12. Common Mistakes

Using tanh in very deep networks without normalization, forgetting output range, using it for probabilities, not scaling targets when using tanh output.

13. Edge Cases / Limitations

Saturation causes vanishing gradients. Outputs cannot exceed $[-1, 1]$, so regression targets need scaling.

14. Variations

Hard tanh, scaled tanh, tanh-shrink, use inside LSTM/GRU gates with sigmoid.

15. Related Topics

Sigmoid is non-zero-centered. ReLU avoids positive-side saturation. LayerNorm can stabilize tanh-based recurrent models.

16. Interview Questions

1. Formula?

$$(e^x - e^{-x}) / (e^x + e^{-x}).$$

2. Range?

$$(-1, 1).$$

3. Derivative?

$$1 - \tanh(x)^2.$$

4. Why better than sigmoid in hidden layers?

Zero-centered output.

5. Main problem?
Saturation and vanishing gradients.
6. Where used today?
RNN/LSTM internals and bounded outputs.
7. Is tanh probability?
No.
8. Output at zero?
Zero.
9. Relation to sigmoid?
 $\tanh(x) = 2\text{sigmoid}(2x) - 1$.
10. Tanh vs ReLU?
Tanh bounded/saturating; ReLU sparse/non-saturating positive side.

17. Practice Tasks

Plot tanh and derivative, train small RNN cell, scale regression target to $[-1, 1]$, compare tanh vs ReLU in MLP, inspect saturation.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
RNN From Scratch	Sequence prediction	NumPy	Sine wave	Sequence fundamentals
Bounded Regressor	Predicts normalized score	PyTorch	Student scores	Output scaling
Activation Lab	Compares sigmoid/tanh/ReLU	Streamlit	Synthetic	Interview demo

19. Quick Revision

Key idea: zero-centered squashing. Formula: $\tanh'(x) = 1 - \tanh^2(x)$. Use bounded hidden states. Trap: saturation. Interview one-liner: tanh is a zero-centered sigmoid-like activation.

20. Final Cheat Sheet

Definition: bounded activation. Input/output: real to $(-1, 1)$. Steps: apply hyperbolic tangent.
Hyperparameters: none. Pros: zero-centered. Cons: vanishing gradients. Best use: recurrent/bounded outputs.

Softmax

1. Overview

Softmax converts a vector of logits into a probability distribution over mutually exclusive classes.

2. Intuition

Softmax turns competing scores into percentages that sum to 1. If one score is much larger, its class receives most probability.

3. Prerequisites

Logits, exponentials, probability distributions, multiclass classification, cross entropy.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Logits	Raw class scores	Stable training input	<code>[2.0, 0.5, -1.0]</code>	Why no softmax before CE
Normalization	Probabilities sum to 1	Mutually exclusive classes	digit class	Multiclass vs multilabel
Temperature	Controls sharpness	Calibration/sampling	LLM decoding	Temperature effect
Numerical stability	Subtract max logit	Prevents overflow	<code>exp(1000)</code>	Stable softmax

5. Algorithm / Working Process

Subtract max logit for stability, exponentiate each shifted logit, divide by sum of exponentials.

6. Mathematical Foundation

```
softmax(z_i) = exp(z_i) / sum_j exp(z_j)
softmax(z_i / T) controls sharpness
```

Lower T makes distribution sharper; higher T makes it flatter.

7. Practical Implementation

```
import torch
import torch.nn.functional as F

logits = torch.tensor([[2.0, 0.5, -1.0]])
probs = F.softmax(logits, dim=1)
pred = probs.argmax(dim=1)
print(probs, pred)
```

8. Code Explanation

`dim=1` applies softmax across classes. `argmax` chooses the highest-probability class. During training, pass logits directly to `CrossEntropyLoss`.

9. Training / Evaluation

Use softmax probabilities for interpretation, calibration, confidence, and top-k metrics. For loss, prefer logits with cross entropy.

10. Complexity and Cost

Cost is $O(K)$ per sample for K classes. For huge vocabularies, softmax can dominate cost.

11. Common Use Cases

Multiclass classification, language model next-token prediction, attention weights, policy distributions in reinforcement learning.

12. Common Mistakes

Using softmax for multilabel classification, applying along wrong dimension, double-softmax before CE loss, ignoring numerical overflow.

13. Edge Cases / Limitations

Can be overconfident, expensive for very large class counts, probabilities are relative to available classes only.

14. Variations

Temperature softmax, sampled softmax, hierarchical softmax, sparsemax, log-softmax.

15. Related Topics

Cross entropy pairs with softmax. Sigmoid is for independent labels. Attention uses softmax to normalize token relevance scores.

16. Interview Questions

1. What does softmax do?
Converts logits to probabilities summing to 1.
2. Formula?
 $\exp(z_i) / \sum \exp(z_j)$.
3. Why subtract max?
Numerical stability.
4. Softmax vs sigmoid?
Mutually exclusive vs independent labels.
5. Should you apply softmax before CrossEntropyLoss?
No.
6. What is temperature?
A sharpness control.

7. Where used in attention?
Normalizes attention scores.
8. What is log-softmax?
Log probabilities computed stably.
9. Why expensive in LLMs?
Vocabulary can be very large.
10. What does high confidence mean?
Relative model confidence, not guaranteed calibration.

17. Practice Tasks

Implement stable softmax, test temperature, compare sigmoid/softmax on multilabel data, compute top-k accuracy, visualize attention weights.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Digit Softmax Classifier	Multiclass prediction	PyTorch	MNIST	Classic baseline
Temperature Demo	Shows distribution sharpness	Python	Synthetic logits	LLM sampling intuition
Calibration Checker	Plots confidence vs accuracy	PyTorch	CIFAR subset	Production relevance

19. Quick Revision

Key idea: logits to class distribution. Formula: $\exp(z_i) / \sum \exp(z_j)$. Use multiclass output and attention. Trap: wrong dimension or double softmax. Interview one-liner: softmax turns competing scores into normalized probabilities.

20. Final Cheat Sheet

Definition: probability normalization. Input/output: vector logits to probabilities. Steps: shift, exp, divide. Hyperparameters: temperature. Metrics: accuracy/top-k/ECE. Pros: interpretable distribution. Cons: overconfidence, expensive for huge vocab. Best use: mutually exclusive choices.

Loss Functions

1. Overview

A loss function measures how wrong a model's prediction is. Training minimizes loss to learn useful parameters.

2. Intuition

Loss is the model's penalty score. Lower loss means predictions align better with targets.

3. Prerequisites

Supervised learning, optimization, probability, regression/classification, gradients.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Objective	Quantity minimized	Defines learning goal	MSE, CE	Metric vs loss
Differentiability	Supports gradients	Needed for backprop	smooth losses	Non-differentiable metrics
Reduction	Mean/sum over batch	Affects scale	mean loss	LR interaction
Task alignment	Match loss to problem	Better learning	CE for classification	Wrong loss choice

5. Algorithm / Working Process

Compute predictions, compare with targets using loss formula, average or sum over batch, call backward to compute gradients.

6. Mathematical Foundation

Common losses:

```
MSE = (1/n) sum_i (y_i - yhat_i)^2
CE = -sum_i y_i log(p_i)
BCE = -[y log(p) + (1-y)log(1-p)]
```

7. Practical Implementation

```
import torch
import torch.nn as nn

reg_pred = torch.tensor([[2.5], [0.0]])
reg_y = torch.tensor([[3.0], [-1.0]])
print(nn.MSELoss()(reg_pred, reg_y))

logits = torch.tensor([[2.0, 0.5], [0.1, 1.3]])
labels = torch.tensor([0, 1])
print(nn.CrossEntropyLoss()(logits, labels))
```

8. Code Explanation

MSE compares continuous values. Cross entropy compares class logits with integer class labels and applies log-softmax internally.

9. Training / Evaluation

Loss guides training, but metrics guide model selection. For classification use accuracy/F1/AUC; for regression use MAE/RMSE/R2.

10. Complexity and Cost

Usually cheap compared with model forward pass. Large-vocabulary cross entropy can be expensive due to softmax.

11. Common Use Cases

Regression, classification, detection, segmentation, language modeling, contrastive learning, reinforcement learning objectives.

12. Common Mistakes

Using MSE for classification, applying softmax before CE, ignoring class imbalance, optimizing a loss misaligned with business metric.

13. Edge Cases / Limitations

Loss may decrease while useful metric worsens. Outliers dominate MSE. CE can be sensitive to noisy labels.

14. Variations

Huber loss, focal loss, Dice loss, contrastive loss, triplet loss, KL divergence, label-smoothed CE.

15. Related Topics

Optimizers minimize loss. Activation choices must match loss. Cross entropy and MSE are common interview favorites.

16. Interview Questions

1. What is a loss function?
A differentiable training objective.
2. Loss vs metric?
Loss optimizes; metric evaluates.
3. MSE use case?
Regression.
4. CE use case?
Classification.
5. Why not accuracy as loss?
It is non-differentiable.

6. What is reduction?
Mean/sum aggregation.
7. How handle imbalance?
Class weights, focal loss, sampling.
8. Why CE with logits?
Numerical stability.
9. What is Huber loss?
Robust regression loss.
10. Can loss be negative?
Some objectives can, but common MSE/CE are non-negative.

17. Practice Tasks

Implement MSE and CE manually, compare reductions, train with wrong loss and observe behavior, add class weights, plot loss vs metric.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Loss Zoo	Compares losses	PyTorch	Synthetic	Strong intuition
Imbalanced Classifier	Uses weighted/focal loss	PyTorch	Credit fraud	Real-world skill
Robust Regressor	MSE vs Huber	sklearn/PyTorch	Housing	Outlier handling

19. Quick Revision

Key idea: quantify error for optimization. Formula: task-specific. Use differentiable proxy for target metric. Trap: loss/activation mismatch. Interview one-liner: loss tells backprop what "better" means.

20. Final Cheat Sheet

Definition: training objective. Input/output: predictions and labels to scalar. Steps: compare, reduce, backprop. Hyperparameters: weights, reduction, margins. Metrics: separate from loss. Pros: drives learning. Cons: proxy may misalign. Best use: model training.

Cross Entropy

1. Overview

Cross entropy measures the difference between a true label distribution and a predicted probability distribution. It is the standard loss for classification and language modeling.

2. Intuition

It heavily penalizes being confidently wrong. Predicting 0.99 for the wrong class is much worse than predicting 0.55.

3. Prerequisites

Probability, log function, softmax, classification, logits.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Negative log likelihood	Penalizes true class probability	Classification objective	$-\log p_{\text{true}}$	CE simplification
Logits	Raw model scores	Stable CE computation	class scores	PyTorch API
One-hot labels	True distribution	CE with classes	$[0, 1, 0]$	Integer labels
Label smoothing	Softens targets	Reduces overconfidence	true class 0.9	Regularization

5. Algorithm / Working Process

Model outputs logits. Softmax converts logits to probabilities. Cross entropy takes negative log probability of the correct class and averages over batch.

6. Mathematical Foundation

For one-hot target y :

$$\text{CE}(y, p) = -\sum_i y_i \log(p_i)$$

For class c :

$$\text{CE} = -\log \text{softmax}(z)_c$$

7. Practical Implementation

```
import torch
import torch.nn as nn

logits = torch.tensor([[3.0, 1.0, 0.2], [0.1, 2.0, 0.3]])
targets = torch.tensor([0, 1])

criterion = nn.CrossEntropyLoss()
loss = criterion(logits, targets)
print(loss.item())
```

8. Code Explanation

Targets are integer class IDs, not one-hot vectors. `CrossEntropyLoss` applies `log_softmax` internally, so raw logits are correct.

9. Training / Evaluation

Use CE for multiclass classification. Track accuracy, top-k accuracy, F1, calibration, and confusion matrix. Add class weights for imbalance.

10. Complexity and Cost

Cost is $O(\text{batch} * \text{classes})$. For LLM vocabularies, CE over many tokens and large vocabularies is a major compute cost.

11. Common Use Cases

Image classification, text classification, token prediction, segmentation, speech recognition, policy learning.

12. Common Mistakes

Applying softmax before CE, passing one-hot labels to PyTorch CE without correct API, ignoring label imbalance, confusing BCE and CE.

13. Edge Cases / Limitations

Sensitive to noisy labels and overconfidence. Can reward calibrated probabilities poorly if metric is ranking-based.

14. Variations

Binary cross entropy, weighted CE, focal loss, label-smoothed CE, KL-divergence distillation loss.

15. Related Topics

Softmax produces probabilities for CE. BCE handles independent labels. Perplexity is exponential of token-level CE in language models.

16. Interview Questions

1. What is cross entropy?
Difference between true and predicted distributions.
2. Formula for one-hot CE?
 $-\log p_{\text{true}}$.
3. Why use logits in PyTorch?
Numerical stability.
4. CE vs MSE for classification?
CE gives better probabilistic gradients.
5. What is label smoothing?
Soft target distribution to reduce overconfidence.

6. What is weighted CE?

Class-weighted loss for imbalance.

7. What is perplexity?

$\exp(\text{cross_entropy})$ for language models.

8. BCE vs CE?

Independent binary labels vs mutually exclusive classes.

9. What happens if predicted true probability is zero?

Loss tends to infinity.

10. Does lower CE always mean better accuracy?

Not always, but often correlated.

17. Practice Tasks

Compute CE manually, train a classifier, add class weights, compare label smoothing, calculate perplexity from token CE.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
CE Classifier	Trains multiclass model	PyTorch	Fashion-MNIST	Core DL
Imbalance CE Study	Weighted vs unweighted CE	PyTorch	Credit fraud	Applied skill
Mini LM Loss	Token CE and perplexity	PyTorch	Tiny Shakespeare	NLP foundation

19. Quick Revision

Key idea: punish low probability on correct class. Formula: $-\log p_{\text{true}}$. Use classification/LMs. Trap: softmax before CE. Interview one-liner: cross entropy trains models to put probability mass on the correct class.

20. Final Cheat Sheet

Definition: classification loss. Input/output: logits and labels to scalar. Steps: log-softmax, select true class, average. Hyperparameters: class weights, label smoothing. Metrics: accuracy/F1/perplexity. Pros: strong gradients. Cons: label-noise sensitivity. Best use: classification.

MSE Loss

1. Overview

Mean Squared Error loss measures average squared difference between predicted and true continuous values. It is the standard baseline loss for regression.

2. Intuition

MSE punishes large errors more than small errors because errors are squared.

3. Prerequisites

Regression, averages, squared error, gradients, outliers.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Squared error	$(y - \hat{y})^2$	Penalizes large mistakes	error 4 -> loss 16	Outlier sensitivity
Mean reduction	Average over samples	Stable scale	batch loss	Sum vs mean
Regression target	Continuous label	Main use case	house price	Classification mismatch
Gradient	Proportional to error	Larger errors update more	$2(\hat{y} - y)$	Optimization behavior

5. Algorithm / Working Process

Predict continuous values, subtract targets, square errors, average them, backpropagate gradient.

6. Mathematical Foundation

```
MSE = (1/n) sum_i (y_i - yhat_i)^2
dMSE/dyhat_i = (2/n)(yhat_i - y_i)
RMSE = sqrt(MSE)
```

7. Practical Implementation

```
import torch
import torch.nn as nn

model = nn.Linear(3, 1)
x = torch.randn(8, 3)
y = torch.randn(8, 1)

pred = model(x)
loss = nn.MSELoss()(pred, y)
loss.backward()
print(loss.item())
```

8. Code Explanation

The model predicts one continuous value per sample. MSE computes the average squared prediction error and supports backpropagation.

9. Training / Evaluation

Use train/validation/test splits. Track MAE, RMSE, R2, residual plots, and outlier behavior. Scale targets when magnitudes are large.

10. Complexity and Cost

Elementwise $O(n)$ over predictions. Very cheap compared with model forward pass.

11. Common Use Cases

Regression, autoencoders, denoising, reconstruction, forecasting, value function approximation.

12. Common Mistakes

Using MSE for classification probabilities, ignoring outliers, comparing MSE across differently scaled targets, not inverse-transforming scaled targets.

13. Edge Cases / Limitations

Very sensitive to outliers. If target noise is heavy-tailed, MAE or Huber may work better.

14. Variations

RMSE, MAE/L1 loss, Huber loss, SmoothL1, weighted MSE, quantile loss.

15. Related Topics

Cross entropy is preferred for classification. Huber is robust to outliers. Gaussian negative log likelihood connects MSE to maximum likelihood with constant variance.

16. Interview Questions

1. Formula?
Average squared error.
2. Why square error?
Penalizes large errors strongly.
3. Gradient?
 $2(\hat{y} - y)/n$.
4. MSE vs MAE?
MSE more outlier-sensitive.
5. MSE vs RMSE?
RMSE is in original target units.
6. Main use case?
Regression.
7. Why bad for classification?
Weak probability/logit gradients compared with CE.
8. Can MSE be negative?
No.

9. What if targets are large?

Scale targets or tune LR.

10. Probabilistic assumption?

Gaussian noise with constant variance.

17. Practice Tasks

Implement MSE manually, compare MSE/MAE on outliers, train linear regression in PyTorch, plot residuals, scale and inverse-scale targets.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
House Price Regressor	Predicts price	PyTorch/sklearn	California housing	Applied regression
Autoencoder	Reconstructs images	PyTorch	MNIST	Representation learning
Forecast Model	Predicts next value	PyTorch	Air passengers	Time-series baseline

19. Quick Revision

Key idea: average squared prediction error. Formula: $(1/n) \sum (y - \hat{y})^2$. Use regression/reconstruction.

Trap: outliers. Interview one-liner: MSE is regression loss that grows quadratically with error.

20. Final Cheat Sheet

Definition: squared-error loss. Input/output: continuous predictions and targets to scalar. Steps: subtract, square, average. Hyperparameters: reduction, weights. Metrics: RMSE, MAE, R2. Pros: simple, differentiable. Cons: outlier-sensitive. Best use: regression with roughly Gaussian noise.

Optimizers

1. Overview

Optimizers update model parameters using gradients to reduce loss. They determine how fast and how reliably a model learns.

2. Intuition

If loss is a landscape, gradients point uphill; optimizers decide how to step downhill without bouncing, crawling, or overshooting.

3. Prerequisites

Gradients, loss functions, learning rate, backpropagation, parameters.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Learning rate	Step size	Most important hyperparameter	$1e-3$	Too high/low
Momentum	Smooths updates	Faster convergence	SGD momentum	Escaping ravines
Adaptive rates	Per-parameter scaling	Handles sparse/noisy gradients	Adam	Adam vs SGD
Weight decay	Penalizes large weights	Regularization	AdamW	L2 vs decoupled decay

5. Algorithm / Working Process

Compute gradients by backpropagation, optionally transform gradients using momentum/adaptive statistics, update parameters, clear gradients before next batch.

6. Mathematical Foundation

SGD:

```
theta := theta - eta * g
```

Momentum:

```
v := beta v + g
theta := theta - eta v
```

Adam:

```
m := beta1*m + (1-beta1)*g
v := beta2*v + (1-beta2)*g^2
theta := theta - eta * m_hat / (sqrt(v_hat) + eps)
```

7. Practical Implementation

```
import torch
import torch.nn as nn

model = nn.Linear(10, 2)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)
```

```
criterion = nn.CrossEntropyLoss()

x = torch.randn(16, 10)
y = torch.randint(0, 2, (16,))

logits = model(x)
loss = criterion(logits, y)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

8. Code Explanation

AdamW uses adaptive moments and decoupled weight decay. The training step follows the standard PyTorch loop: forward, loss, zero gradients, backward, update.

9. Training / Evaluation

Tune learning rate first. Use schedulers for longer training. AdamW is a strong default for deep learning; SGD with momentum can generalize well in vision.

10. Complexity and Cost

SGD stores minimal state. Adam stores first and second moments, roughly doubling parameter-state memory.

11. Common Use Cases

Training MLPs, CNNs, Transformers, fine-tuning, large-scale pretraining, reinforcement learning.

12. Common Mistakes

Learning rate too high, forgetting `zero_grad`, using Adam weight decay incorrectly instead of AdamW, not scheduling LR, comparing optimizers without tuning LR.

13. Edge Cases / Limitations

Adam can overfit or generalize worse than SGD in some settings. Large models need careful LR warmup and gradient clipping.

14. Variations

SGD, Momentum, RMSProp, Adam, AdamW, Adagrad, Lion, LAMB, Adafactor.

15. Related Topics

Backpropagation supplies gradients. Weight initialization affects optimizer stability. Mixed precision may require gradient scaling.

16. Interview Questions

1. What does an optimizer do?
Updates parameters using gradients.
2. What is learning rate?
Step size.
3. SGD vs Adam?
SGD uses raw gradients; Adam adapts per parameter.
4. Why momentum?
Reduces oscillation and speeds learning.
5. AdamW vs Adam with L2?
AdamW decouples weight decay from gradient update.
6. What if LR is too high?
Divergence/oscillation.
7. What if LR is too low?
Slow training/stuck.
8. Why use LR warmup?
Stabilizes early training.
9. What is gradient clipping?
Limits gradient norm.
10. Which optimizer for Transformers?
AdamW is common.

17. Practice Tasks

Train same MLP with SGD/Adam, plot loss curves, tune LR, add scheduler, compare Adam and AdamW.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Optimizer Race	Compares convergence	PyTorch	MNIST	Training insight
LR Finder	Finds good LR range	PyTorch	Any	Practical tooling
AdamW Fine-Tuner	Fine-tunes classifier	Hugging Face	IMDb	Modern workflow

19. Quick Revision

Key idea: gradients become parameter updates. Formula: $\theta = \theta - lr * grad$. Use all neural training. Trap: bad LR. Interview one-liner: optimizers decide how the model moves through the loss landscape.

20. Final Cheat Sheet

Definition: parameter update rule. Input/output: gradients to new parameters. Steps: compute grad, transform, update. Hyperparameters: LR, momentum, betas, weight decay. Metrics: validation loss. Pros: enables learning. Cons: sensitive tuning. Best use: all trainable models.

Dropout

1. Overview

Dropout is a regularization technique that randomly disables neurons during training to reduce overfitting.

2. Intuition

It prevents neurons from relying too heavily on specific other neurons, forcing the network to learn more robust features.

3. Prerequisites

Overfitting, regularization, neural networks, training vs inference mode.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Drop probability	Fraction zeroed	Controls regularization	<code>p=0.5</code>	Too much dropout
Inverted dropout	Scales during training	Keeps expected activation	PyTorch default	Train/eval behavior
Model averaging	Approximate ensemble	Improves generalization	subnetworks	Intuition
Eval mode	Disables dropout	Deterministic inference	<code>model.eval()</code>	Common bug

5. Algorithm / Working Process

During training, sample a Bernoulli mask, zero selected activations, scale remaining activations. During inference, use all activations.

6. Mathematical Foundation

```
m_i ~ Bernoulli(1-p)
a'_i = (m_i * a_i) / (1-p)
E[a'_i] = a_i
```

7. Practical Implementation

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(100, 64),
    nn.ReLU(),
    nn.Dropout(p=0.3),
```

```
nn.Linear(64, 10),  
)
```

8. Code Explanation

Dropout is placed after activation in the hidden layer. It only changes behavior in training mode.

9. Training / Evaluation

Use dropout when train loss is much lower than validation loss. Tune `p`; common values are `0.1` to `0.5`. Always use `model.eval()` for validation/inference.

10. Complexity and Cost

Very low compute overhead. No trainable parameters. Training can need more epochs because the task is noisier.

11. Common Use Cases

MLPs, classification heads, Transformers attention/MLP dropout, small-to-medium datasets.

12. Common Mistakes

Forgetting eval mode, using too much dropout, applying dropout where BatchNorm already regularizes strongly, expecting it to fix data leakage.

13. Edge Cases / Limitations

Less useful with huge datasets, can hurt underfitting models, spatial dropout variants may be better for CNN feature maps.

14. Variations

DropConnect, SpatialDropout, AlphaDropout, attention dropout, stochastic depth.

15. Related Topics

Weight decay also regularizes. BatchNorm adds noise-like regularization. Stochastic depth drops layers instead of neurons.

16. Interview Questions

1. What is dropout?
Randomly zeroing activations during training.
2. Why use it?
Reduce overfitting.
3. Is dropout active in inference?
No.

4. What is inverted dropout?
Scaling kept activations during training.
5. What does p mean?
Probability of dropping.
6. Too much dropout causes?
Underfitting.
7. Dropout vs weight decay?
Activation noise vs parameter penalty.
8. Where place dropout?
Usually after activation or in architecture-defined spots.
9. Does dropout add parameters?
No.
10. Why call `model.eval()`?
Disable dropout for deterministic inference.

17. Practice Tasks

Train MLP with/without dropout, vary p , compare train-validation gap, inspect stochastic outputs in train mode, add dropout to Transformer block.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Overfit Fixer	Demonstrates dropout	PyTorch	Small MNIST subset	Regularization proof
Dropout Tuner	Searches p	PyTorch	Fashion-MNIST	Hyperparameter skill
MC Dropout Demo	Estimates uncertainty	PyTorch	Synthetic regression	Research angle

19. Quick Revision

Key idea: random neuron removal during training. Formula: $a' = m * a / (1 - p)$. Use overfitting control. Trap: eval mode. Interview one-liner: dropout regularizes by training many random subnetworks.

20. Final Cheat Sheet

Definition: stochastic regularizer. Input/output: activations to masked activations. Steps: mask, scale, train; disable at eval. Hyperparameters: p . Metrics: validation gap. Pros: reduces overfitting. Cons: can underfit. Best use: dense/Transformer layers.

Batch Normalization

1. Overview

Batch normalization normalizes layer activations using batch statistics, then learns scale and shift parameters. It stabilizes and often speeds training.

2. Intuition

It keeps intermediate activations in a healthier range so later layers do not constantly chase changing input distributions.

3. Prerequisites

Mean, variance, mini-batches, neural layers, train/eval modes.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Batch stats	Mean/variance over mini-batch	Normalization source	channel mean	Train/eval difference
Learnable gamma/beta	Scale and shift	Restores expressiveness	affine transform	Why not fixed normalize
Running stats	Moving averages	Used at inference	running_mean	Small batch issue
Placement	Before/after activation	Architecture-dependent	Conv-BN-ReLU	Practical design

5. Algorithm / Working Process

Compute batch mean and variance, normalize activations, apply learned scale **gamma** and shift **beta**, update running statistics for inference.

6. Mathematical Foundation

```
mu_B = mean(x_B)
sigma_B^2 = var(x_B)
x_hat = (x - mu_B) / sqrt(sigma_B^2 + eps)
y = gamma * x_hat + beta
```

7. Practical Implementation

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(100, 64),
    nn.BatchNorm1d(64),
    nn.ReLU(),
    nn.Linear(64, 10),
)
```

8. Code Explanation

`BatchNorm1d(64)` normalizes the 64 hidden features across the batch and learns one scale/shift pair per feature.

9. Training / Evaluation

Use sufficiently large batches. Switch to `model.eval()` for validation/inference so running statistics are used.

10. Complexity and Cost

Adds small compute and memory for statistics plus $2 * \text{features}$ trainable parameters.

11. Common Use Cases

CNNs, MLPs, residual networks, fast supervised vision training.

12. Common Mistakes

Tiny batch sizes, forgetting eval mode, mixing train/eval stats, using BatchNorm carelessly in sequence models with variable lengths.

13. Edge Cases / Limitations

Performs poorly with very small batches, distributed training needs SyncBatchNorm, inference depends on good running statistics.

14. Variations

BatchNorm1d/2d/3d, SyncBatchNorm, Batch Renormalization, Ghost BatchNorm.

15. Related Topics

LayerNorm normalizes per sample and works well in Transformers. GroupNorm handles small batches in vision. Weight initialization and residuals also stabilize training.

16. Interview Questions

1. What is BatchNorm?
Batch-stat normalization with learnable affine parameters.
2. Formula?
 $(x - \mu) / \sqrt{\text{var} + \text{eps}} * \text{gamma} + \text{beta}.$
3. Why gamma/beta?
Restore representational flexibility.
4. Train vs eval?
Train uses batch stats; eval uses running stats.
5. Problem with small batch?
Noisy statistics.
6. Where common?
CNNs/ResNets.
7. Does it regularize?
Slightly, due to batch noise.

8. BatchNorm vs LayerNorm?

Batch vs feature/sample statistics.

9. What is SyncBatchNorm?

Stats synchronized across devices.

10. Does BatchNorm remove need for initialization?

No, but it helps stability.

17. Practice Tasks

Add BatchNorm to MLP, compare convergence, test batch sizes, inspect running stats, intentionally forget eval mode and observe validation noise.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
BN Speed Test	Measures convergence	PyTorch	MNIST	Training optimization
Small Batch Study	BN vs GroupNorm	PyTorch	CIFAR subset	Practical insight
Running Stats Debugger	Visualizes stats	PyTorch hooks	Any CNN	Production debugging

19. Quick Revision

Key idea: normalize using batch stats. Formula: $(x - \mu_B) / \sqrt{\text{var}_B + \epsilon}$. Use CNNs/MLPs. Trap: train/eval mismatch. Interview one-liner: BatchNorm stabilizes activations using mini-batch statistics and learned affine recovery.

20. Final Cheat Sheet

Definition: batch-stat normalization layer. Input/output: activations to normalized activations. Steps: mean, variance, normalize, scale, shift. Hyperparameters: eps, momentum. Metrics: convergence/validation. Pros: faster training. Cons: small-batch weakness. Best use: CNNs.

Layer Normalization

1. Overview

Layer normalization normalizes features within each sample rather than across the batch. It is standard in Transformers and sequence models.

2. Intuition

Each example normalizes itself, so behavior does not depend on which other examples are in the batch.

3. Prerequisites

Mean/variance, tensors, sequence models, Transformers, normalization.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Per-sample stats	Mean/variance over features	Batch-size independent	token hidden vector	Transformer fit
Gamma/beta	Learnable affine	Restores scale/shift	hidden dimension	Parameter count
Pre-LN/Post-LN	Placement around residual block	Affects deep stability	Transformer blocks	Modern design
No running stats	Same train/eval behavior	Stable inference	no batch dependency	BN comparison

5. Algorithm / Working Process

For each sample/token, compute mean and variance across hidden features, normalize, then apply learned scale and shift.

6. Mathematical Foundation

```
mu = (1/H) sum_j x_j
sigma^2 = (1/H) sum_j (x_j - mu)^2
y_j = gamma_j * (x_j - mu) / sqrt(sigma^2 + eps) + beta_j
```

7. Practical Implementation

```
import torch
import torch.nn as nn

x = torch.randn(2, 5, 768) # batch, tokens, hidden
ln = nn.LayerNorm(768)
y = ln(x)
print(y.shape)
```

8. Code Explanation

LayerNorm normalizes the last dimension of each token independently. This matches Transformer hidden-state layout.

9. Training / Evaluation

Works well with small or variable batch sizes. Pre-LN Transformers train deeper more stably than original Post-LN designs.

10. Complexity and Cost

Cost is $O(\text{number_of_elements})$ plus mean/variance reductions. Adds $2 * \text{hidden_dim}$ parameters.

11. Common Use Cases

Transformers, LLMs, RNNs, small-batch settings, reinforcement learning networks.

12. Common Mistakes

Normalizing wrong dimension, confusing with BatchNorm, removing affine parameters unnecessarily, using BatchNorm in Transformer blocks by habit.

13. Edge Cases / Limitations

LayerNorm can be slower than simpler elementwise ops, and in some vision CNNs BatchNorm/GroupNorm may work better.

14. Variations

RMSNorm, ScaleNorm, Pre-LN, Post-LN, Sandwich-LN.

15. Related Topics

BatchNorm uses batch stats. RMSNorm removes mean subtraction. Transformers rely on LayerNorm with residual connections.

16. Interview Questions

1. What is LayerNorm?
Normalization across features within each sample.
2. Why used in Transformers?
Batch-independent and stable for sequences.
3. Formula?
 $(x - \mu_{\text{features}}) / \sqrt{\text{var_features} + \text{eps}}$.
4. Train vs eval?
Same statistics; no running stats.
5. LayerNorm vs BatchNorm?
Per-sample features vs batch dimension.
6. What are gamma/beta?
Learnable scale and shift.
7. What is Pre-LN?
LayerNorm before sublayer.

8. Why Pre-LN?
Better gradient flow in deep Transformers.
9. What is RMSNorm?
Normalizes by root mean square only.
10. Does LayerNorm depend on batch size?
No.

17. Practice Tasks

Implement LayerNorm manually, compare with PyTorch, add Pre-LN Transformer block, test batch size 1, inspect feature mean/variance.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
LayerNorm From Scratch	Manual implementation	PyTorch	Random tensors	Core DL
Mini Transformer Block	Uses Pre-LN	PyTorch	Tiny text	LLM foundation
Norm Comparator	BN vs LN vs GN	PyTorch	CIFAR/text	Architecture insight

19. Quick Revision

Key idea: normalize features per sample. Formula: feature mean/variance. Use Transformers. Trap: wrong normalized dimension. Interview one-liner: LayerNorm stabilizes each token's hidden vector independently of the batch.

20. Final Cheat Sheet

Definition: per-sample feature normalization. Input/output: hidden vector to normalized vector. Steps: mean, variance, normalize, affine. Hyperparameters: eps, normalized shape. Metrics: training stability. Pros: batch-independent. Cons: not always best for CNNs. Best use: Transformers.

Weight Initialization

1. Overview

Weight initialization chooses starting parameter values before training. Good initialization preserves signal and gradient scale across layers.

2. Intuition

If weights start too small, signals vanish; too large, signals explode. Good initialization gives training a stable starting point.

3. Prerequisites

Variance, neural layers, activations, gradients, vanishing/exploding gradients.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Symmetry breaking	Different neurons need different weights	Enables diverse learning	random init	Why not all zeros
Fan-in/fan-out	Input/output connections	Sets variance scale	Linear layer dimensions	Xavier/He formulas
Activation-aware init	Match init to activation	Preserves variance	He for ReLU	Practical pairing
Bias init	Usually zeros	Simple and stable	output bias	Class prior trick

5. Algorithm / Working Process

Choose initialization based on layer type and activation, sample weights from normal/uniform distribution with calculated variance, initialize biases simply, then train.

6. Mathematical Foundation

Xavier/Glorot:

$$\text{Var}(W) = 2 / (\text{fan_in} + \text{fan_out})$$

He/Kaiming for ReLU:

$$\text{Var}(W) = 2 / \text{fan_in}$$

7. Practical Implementation

```
import torch.nn as nn

model = nn.Sequential(nn.Linear(100, 64), nn.ReLU(), nn.Linear(64, 10))

for module in model.modules():
    if isinstance(module, nn.Linear):
        nn.init.kaiming_normal_(module.weight, nonlinearity="relu")
        nn.init.zeros_(module.bias)
```

8. Code Explanation

Kaiming initialization matches ReLU hidden layers. Biases are set to zero because random weights already break symmetry.

9. Training / Evaluation

Bad initialization shows up as stagnant loss, NaNs, vanishing gradients, or exploding activations. Inspect activation/gradient statistics early.

10. Complexity and Cost

One-time setup cost proportional to number of parameters. No inference cost.

11. Common Use Cases

Training deep MLPs/CNNs/Transformers, transfer learning heads, custom layers, research experiments.

12. Common Mistakes

Initializing all weights to zero, using Xavier for very deep ReLU nets, reinitializing pretrained weights accidentally, ignoring output-layer bias for imbalanced data.

13. Edge Cases / Limitations

Normalization and residuals reduce sensitivity but do not remove it. Special architectures may need custom initialization.

14. Variations

Xavier uniform/normal, He uniform/normal, orthogonal init, normal init, truncated normal, LSUV, zero-init residual branches.

15. Related Topics

ReLU pairs with He. Tanh/sigmoid pair with Xavier. Residual networks often initialize residual branch carefully. NTK theory studies infinite-width initialization.

16. Interview Questions

1. Why initialize randomly?
Break symmetry.
2. Why not all zeros?
Neurons learn identical features.
3. What is fan-in?
Number of input connections.
4. Xavier formula?
Variance $2/(\text{fan_in} + \text{fan_out})$.
5. He formula?
Variance $2/\text{fan_in}$.

6. He for which activation?
ReLU-like activations.
7. What if weights too large?
Exploding activations/gradients.
8. What if too small?
Vanishing signals/gradients.
9. Bias init default?
Usually zero.
10. Should pretrained models be reinitialized?
Usually no, except new heads.

17. Practice Tasks

Compare zero/random/Xavier/He, plot activation variance by layer, inspect gradient norms, initialize output bias using class prior, test deep MLP stability.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Init Lab	Compares initialization schemes	PyTorch	MNIST	Training diagnostics
Deep Signal Tracker	Plots layer variances	PyTorch	Random data	Research intuition
Imbalanced Bias Init	Faster rare-class training	PyTorch	Fraud data	Applied trick

19. Quick Revision

Key idea: stable starting weights. Formula: He $2/\text{fan_in}$, Xavier $2/(\text{fan_in}+\text{fan_out})$. Use before training. Trap: zero weights. Interview one-liner: initialization keeps signals and gradients from dying or exploding at step zero.

20. Final Cheat Sheet

Definition: parameter start strategy. Input/output: layer shapes to initial weights. Steps: choose scheme, sample, train. Hyperparameters: distribution, gain. Metrics: activation/gradient stats. Pros: stable training. Cons: architecture-specific. Best use: all neural training.

Vanishing Gradients

1. Overview

Vanishing gradients occur when gradients become extremely small as they propagate backward, making early layers learn slowly or not at all.

2. Intuition

If each layer multiplies the gradient by a number less than 1, after many layers the signal becomes nearly zero.

3. Prerequisites

Backpropagation, chain rule, activation derivatives, deep networks.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Chain multiplication	Many derivatives multiply	Can shrink signal	0.5^{50}	Root cause
Saturation	Derivatives near zero	Common with sigmoid/tanh	large logits	Activation choice
Early layers	Far from loss	Learn slowest	first CNN layers	Gradient flow
Remedies	Architectural/training fixes	Enables deep models	ReLU, residuals	Practical answer

5. Algorithm / Working Process

During backpropagation, gradients pass through each layer's derivative. If derivatives or weight singular values are small repeatedly, gradients reaching early layers vanish.

6. Mathematical Foundation

For composed functions:

$$dL/dx = dL/da_L * \text{product}_l da_l/da_{\{l-1\}}$$

If each factor has magnitude < 1 , the product shrinks exponentially with depth.

7. Practical Implementation

```
import torch
import torch.nn as nn

model = nn.Sequential(*[
    layer for _ in range(10)
    for layer in (nn.Linear(32, 32), nn.Sigmoid())
])
x = torch.randn(4, 32)
loss = model(x).mean()
```

```
loss.backward()

for name, p in model.named_parameters():
    if p.grad is not None:
        print(name, p.grad.norm().item())
```

8. Code Explanation

A deep sigmoid network often shows small gradients in early layers because sigmoid derivatives are at most 0.25.

9. Training / Evaluation

Detect by logging gradient norms per layer. Fix with ReLU/GELU, residual connections, normalization, proper initialization, shorter paths, and LSTM/GRU for old RNN setups.

10. Complexity and Cost

Detection has small logging overhead. Fixes like residuals and normalization add minor compute but greatly improve trainability.

11. Common Use Cases

Debugging deep MLPs, RNNs, old sigmoid/tanh networks, very deep CNNs.

12. Common Mistakes

Only increasing learning rate, using sigmoid everywhere, not checking gradient norms, confusing vanishing gradients with overfitting.

13. Edge Cases / Limitations

Residuals reduce but do not eliminate all optimization issues. Very long sequence credit assignment remains hard.

14. Variations

Vanishing activations, vanishing updates under adaptive optimizers, long-term dependency failure in RNNs.

15. Related Topics

Exploding gradients are the opposite instability. ReLU, residual connections, normalization, and initialization are standard remedies.

16. Interview Questions

1. What are vanishing gradients?
Gradients become too small for learning.
2. Main cause?
Chain rule products below 1.

3. Which activations worsen it?
Sigmoid/tanh in saturated regions.
4. How detect?
Gradient norm logging.
5. How fix?
ReLU, residuals, normalization, good init.
6. Why RNNs suffer?
Repeated temporal multiplication.
7. Does Adam solve it fully?
No.
8. Why are residuals helpful?
Provide shorter gradient paths.
9. What is sigmoid max derivative?
0.25.
10. Vanishing gradient vs underfitting?
Vanishing gradient is an optimization failure causing weak learning.

17. Practice Tasks

Train deep sigmoid vs ReLU network, plot gradient norms, add residuals, test Xavier/He init, reproduce RNN long-dependency failure.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Gradient Flow Lab	Compares architectures	PyTorch	Synthetic	Deep debugging
RNN Memory Test	Tests long dependencies	PyTorch	Copy task	Sequence insight
ResNet Fix Demo	Adds skips to deep MLP	PyTorch	MNIST	Architecture skill

19. Quick Revision

Key idea: gradients shrink backward. Formula: product of small derivatives. Use fixes in deep models. Trap: just raising LR. Interview one-liner: vanishing gradients make early layers nearly stop learning.

20. Final Cheat Sheet

Definition: tiny backward signals. Input/output: loss gradient becomes near zero. Steps: chain rule shrinkage. Hyperparameters: init, activation, depth. Metrics: gradient norms. Pros: none. Cons: training failure. Best fix: residuals, ReLU/GELU, normalization, good init.

Exploding Gradients

1. Overview

Exploding gradients occur when gradients become extremely large during backpropagation, causing unstable updates, NaNs, or divergence.

2. Intuition

If each layer multiplies the gradient by a number greater than 1, the backward signal can blow up exponentially.

3. Prerequisites

Backpropagation, gradient descent, matrix norms, learning rate, deep/recurrent networks.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Large gradient norm	Huge update direction	Can destroy weights	NaN loss	Detection
Chain amplification	Products above 1	Exponential growth	RNN recurrence	Root cause
Clipping	Limits gradient norm/value	Stabilizes training	<code>clip_grad_norm_</code>	Practical fix
Initialization	Controls signal scale	Prevents early explosion	He/Xavier	Prevention

5. Algorithm / Working Process

During backpropagation, gradients multiply through layers or time steps. Large derivatives or weight norms amplify gradients. Optimizer step then changes weights too much.

6. Mathematical Foundation

$$dL/dx = dL/da_L * \text{product}_1 J_1$$

If Jacobian norms are repeatedly > 1 , gradient norm can grow exponentially.

Gradient clipping:

$$g := g * \min(1, \text{max_norm} / ||g||)$$

7. Practical Implementation

```
import torch

loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

8. Code Explanation

After gradients are computed, `clip_grad_norm_` rescales them if their global norm exceeds `1.0`, preventing oversized updates.

9. Training / Evaluation

Watch for NaNs, sudden loss spikes, huge gradient norms. Fix with lower LR, gradient clipping, normalization, good initialization, residual connections, and stable loss computation.

10. Complexity and Cost

Gradient clipping adds a pass over parameters, usually cheap compared with training.

11. Common Use Cases

RNN training, LLM training/fine-tuning, reinforcement learning, unstable deep models.

12. Common Mistakes

Clipping before `backward`, clipping too low, ignoring NaNs, using high LR, not checking data scaling.

13. Edge Cases / Limitations

Clipping hides symptoms if model/loss/data are broken. It stabilizes updates but may slow learning if threshold is too small.

14. Variations

Norm clipping, value clipping, adaptive clipping, loss scaling checks, spectral normalization.

15. Related Topics

Vanishing gradients are the opposite. Mixed precision can overflow without gradient scaling. Initialization and normalization reduce risk.

16. Interview Questions

1. What are exploding gradients?
Gradients become too large.
2. Symptoms?
NaN loss, divergence, huge updates.

3. Main cause?
Chain products with norms above 1.
4. How detect?
Log gradient norms.
5. Main fix?
Gradient clipping.
6. Formula for norm clipping?
Scale by $\text{max_norm} / ||g||$ if needed.
7. Other fixes?
Lower LR, init, normalization.
8. Why common in RNNs?
Repeated recurrence over time.
9. Does clipping change gradient direction?
Norm clipping preserves direction.
10. Can mixed precision worsen it?
Overflow can appear if scaling is mishandled.

17. Practice Tasks

Create unstable RNN, add clipping, log gradient norms, test LR sensitivity, catch NaNs in training loop.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Gradient Clipping Demo	Stabilizes RNN	PyTorch	Character text	Sequence training
NaN Debugger	Finds bad batches	PyTorch	Any	Production skill
LR Stress Test	Maps stable LR range	PyTorch	MNIST	Optimizer intuition

19. Quick Revision

Key idea: gradients blow up through depth/time. Formula: Jacobian norm product. Use clipping. Trap: clipping bad data bugs. Interview one-liner: exploding gradients make updates too large to train stably.

20. Final Cheat Sheet

Definition: oversized gradients. Input/output: backward signal becomes huge. Steps: amplification, giant update, divergence. Hyperparameters: max norm, LR. Metrics: gradient norm, NaNs. Pros: none. Cons: unstable training. Best fix: clipping plus stable setup.

Residual Connections

1. Overview

Residual connections add a block's input to its output, allowing networks to learn residual functions. They made very deep networks practical.

2. Intuition

Instead of forcing a block to learn a full transformation, it learns a correction to the input.

3. Prerequisites

Neural blocks, tensor shapes, backpropagation, vanishing gradients.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Identity path	Direct input route	Helps gradient flow	$x + F(x)$	Why ResNet works
Residual function	Learned correction	Easier optimization	refine features	Degradation problem
Shape matching	Same tensor shape needed	Addition requirement	projection shortcut	Practical detail
Deep trainability	More layers possible	Better representation	ResNet-50	Vanishing gradients

5. Algorithm / Working Process

Pass input through a block $F(x)$, optionally project x to matching shape, add input and block output, then apply activation depending on architecture.

6. Mathematical Foundation

$$y = F(x, W) + x$$
$$dL/dx = dL/dy * (dF/dx + I)$$

The identity term I provides a direct gradient path.

7. Practical Implementation

```
import torch
import torch.nn as nn

class ResidualMLPBlock(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.block = nn.Sequential(nn.Linear(dim, dim), nn.ReLU(), nn.Linear(dim, dim))

    def forward(self, x):
        return torch.relu(x + self.block(x))
```

8. Code Explanation

The block output has the same dimension as input, so addition is valid. The model learns $F(x)$ as a residual correction.

9. Training / Evaluation

Useful for deep networks. If dimensions change, use projection shortcuts. Combine with normalization and good initialization.

10. Complexity and Cost

Addition cost is tiny. Projection shortcuts add parameters when dimensions differ.

11. Common Use Cases

ResNets, Transformers, diffusion U-Nets, deep MLPs, segmentation networks.

12. Common Mistakes

Adding tensors with mismatched shapes, assuming residuals fix all training issues, placing normalization poorly, using residuals where output semantics should not preserve input.

13. Edge Cases / Limitations

Very deep residual networks still need normalization and initialization. Residual paths can preserve unwanted features.

14. Variations

Pre-activation residual blocks, bottleneck residual blocks, gated residuals, dense connections, highway networks.

15. Related Topics

Skip connections are a broader category. LayerNorm and residuals form Transformer blocks. Vanishing gradients motivate residuals.

16. Interview Questions

1. What is a residual connection?

Adding input to block output.

2. Formula?

$$y = F(x) + x.$$

3. Why helpful?

Improves gradient flow and optimization.

4. What if shapes differ?

Use projection shortcut.

5. ResNet motivation?

Solve degradation in very deep nets.

6. Residual vs plain network?
Residual learns correction.
7. Does it add many parameters?
No, unless projection is used.
8. Where used in Transformers?
Around attention and MLP sublayers.
9. Why identity helps gradients?
Direct derivative path.
10. Is residual same as skip connection?
It is a type of skip connection.

17. Practice Tasks

Implement residual MLP, compare 20-layer plain vs residual, add projection for dimension change, inspect gradient norms, build mini ResNet block.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Deep MLP Rescue	Residual vs plain	PyTorch	MNIST	Optimization insight
Mini ResNet	Image classifier	PyTorch	CIFAR-10	CV baseline
Transformer Block	Residual attention block	PyTorch	Tiny text	LLM foundation

19. Quick Revision

Key idea: learn correction $F(x)$ plus identity. Formula: $y=F(x)+x$. Use deep networks. Trap: shape mismatch. Interview one-liner: residual connections create direct information and gradient highways.

20. Final Cheat Sheet

Definition: input-added block. Input/output: same-shaped tensors. Steps: transform, add, activate/norm. Hyperparameters: projection, block depth. Metrics: trainability. Pros: deep stable training. Cons: shape constraints. Best use: deep architectures.

Skip Connections

1. Overview

Skip connections route information from one layer to a later layer, bypassing intermediate operations. Residual connections and U-Net concatenations are common examples.

2. Intuition

They let the model reuse earlier features directly instead of forcing every detail through all intermediate layers.

3. Prerequisites

Tensor shapes, neural network blocks, concatenation/addition, gradients.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Additive skip	Adds tensors	Residual learning	ResNet	Shape equality
Concatenative skip	Concat channels/features	Preserves detail	U-Net	Memory cost
Long skip	Connects distant layers	Multi-scale features	encoder to decoder	Segmentation
Gradient route	Shorter backward path	Easier training	deep nets	Vanishing fix

5. Algorithm / Working Process

Store an earlier activation, process through later layers, then combine stored activation with later activation using addition or concatenation.

6. Mathematical Foundation

Additive:

$$y = F(x) + x$$

Concatenative:

$$y = \text{concat}(F(x), x)$$

7. Practical Implementation

```
import torch
import torch.nn as nn

class SkipConcatBlock(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.f = nn.Linear(dim, dim)
        self.out = nn.Linear(2 * dim, dim)

    def forward(self, x):
```

```
h = torch.relu(self.f(x))  
return self.out(torch.cat([x, h], dim=-1))
```

8. Code Explanation

The original input and transformed feature are concatenated, so the output layer receives both raw and processed information.

9. Training / Evaluation

Use additive skips for same-dimensional deep blocks. Use concatenative skips when preserving local/detail features matters, such as segmentation.

10. Complexity and Cost

Additive skips are cheap. Concatenation increases feature dimension and downstream compute/memory.

11. Common Use Cases

ResNets, U-Net, DenseNet, Transformers, diffusion models, encoder-decoder networks.

12. Common Mistakes

Confusing addition and concatenation, ignoring shape alignment, memory blow-up with many concat skips, skipping normalization in deep blocks.

13. Edge Cases / Limitations

Skip connections can leak low-level noise or make model rely too much on shallow features. Concats can be memory-heavy.

14. Variations

Residual skip, dense skip, highway gate, U-Net skip, cross-attention skip, feature pyramid connections.

15. Related Topics

Residual connections are additive skips. U-Net uses encoder-decoder skips. DenseNet concatenates all previous features.

16. Interview Questions

1. What is a skip connection?
A connection bypassing one or more layers.
2. Why use it?
Preserve information and improve gradients.
3. Add vs concat?
Add keeps dimension; concat increases dimension.

4. Residual connection type?
Additive skip.
5. U-Net skip type?
Usually concatenative.
6. Shape requirement for addition?
Same shape.
7. Shape effect of concat?
Feature/channel dimension grows.
8. Where in Transformers?
Around attention and MLP.
9. Do skips add parameters?
Not by themselves.
10. Limitation?
Memory/shape complexity, possible feature leakage.

17. Practice Tasks

Implement add and concat skips, build small U-Net block, compare gradients, measure memory, debug shape mismatch.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
U-Net Mini	Segment simple images	PyTorch	Oxford Pets subset	CV project
Dense Skip Classifier	Tests concat features	PyTorch	MNIST	Architecture study
Skip Debugger	Visualizes tensor shapes	PyTorch hooks	Any	Engineering skill

19. Quick Revision

Key idea: bypass layers. Formula: add or concat. Use deep and encoder-decoder models. Trap: shape mismatch. Interview one-liner: skip connections preserve information and shorten gradient paths.

20. Final Cheat Sheet

Definition: bypass connection. Input/output: earlier activation plus later activation. Steps: save, process, combine. Hyperparameters: add/concat/projection. Metrics: accuracy, memory. Pros: trainability/detail. Cons: shape and memory cost. Best use: deep models, U-Net, Transformers.

Normalization Variants

1. Overview

Normalization variants stabilize training by controlling activation distributions. Different variants choose different dimensions for computing statistics.

2. Intuition

Normalization keeps activations from becoming too shifted or scaled, like keeping every layer's input in a workable range.

3. Prerequisites

Mean, variance, tensors, batch/channel/feature dimensions, neural training.

4. Core Concepts

Variant	Stats over	Why it matters	Example	Interview angle
BatchNorm	Batch dimension	Good for CNN large batches	ResNet	Train/eval stats
LayerNorm	Feature dimension per sample	Good for Transformers	LLM block	Batch independent
InstanceNorm	Spatial per instance/channel	Style normalization	style transfer	CV use
GroupNorm	Channel groups	Good for small batches	detection	BN alternative
RMSNorm	RMS only	Cheaper LN-like	LLMs	Mean subtraction removed

5. Algorithm / Working Process

Choose axes, compute mean/variance or RMS, normalize, apply learnable scale and sometimes shift.

6. Mathematical Foundation

General normalization:

```
x_hat = (x - mean_axes(x)) / sqrt(var_axes(x) + eps)
y = gamma * x_hat + beta
```

RMSNorm:

```
y = gamma * x / sqrt(mean(x^2) + eps)
```

7. Practical Implementation

```
import torch.nn as nn

bn = nn.BatchNorm2d(64)
gn = nn.GroupNorm(num_groups=8, num_channels=64)
ln = nn.LayerNorm(768)
```

8. Code Explanation

BatchNorm is for channel maps with good batch size, GroupNorm for small-batch vision, and LayerNorm for hidden vectors such as Transformer states.

9. Training / Evaluation

Pick normalization by architecture and batch size. CNN large batch: BatchNorm. Transformer: LayerNorm/RMSNorm. Small-batch vision: GroupNorm.

10. Complexity and Cost

All variants are roughly linear in tensor size. Memory and reduction axes differ. BatchNorm stores running stats; LayerNorm and GroupNorm usually do not.

11. Common Use Cases

Vision classification, object detection, segmentation, Transformers, LLMs, style transfer.

12. Common Mistakes

Using BatchNorm with batch size 1, normalizing wrong dimension, forgetting eval mode for BN, assuming all norms are interchangeable.

13. Edge Cases / Limitations

Normalization can harm some tasks where scale carries meaning. BN can fail under domain shift if running stats mismatch production data.

14. Variations

BatchNorm, LayerNorm, InstanceNorm, GroupNorm, RMSNorm, WeightNorm, SpectralNorm, Filter Response Norm.

15. Related Topics

BatchNorm and LayerNorm have dedicated sections. Weight initialization and residual connections also stabilize deep training.

16. Interview Questions

1. Why normalize activations?
Improve stability and optimization.

2. BN stats over what?
Batch, often per channel.
3. LN stats over what?
Features per sample.
4. GN useful when?
Small-batch vision.
5. InstanceNorm used where?
Style transfer/generation.
6. RMSNorm difference?
Uses RMS, often no mean subtraction.
7. Which for Transformers?
LayerNorm/RMSNorm.
8. Which has running stats?
BatchNorm.
9. BN issue with batch size 1?
Unreliable stats.
10. Does normalization add parameters?
Usually scale and shift.

17. Practice Tasks

Compare BN/LN/GN on small batches, implement RMSNorm, inspect normalized axes, benchmark training stability, test eval behavior.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Norm Selector Study	Compares norms	PyTorch	CIFAR-10	Architecture tuning
RMSNorm Transformer	Mini language model	PyTorch	Tiny Shakespeare	LLM relevance
Small-Batch Detector	BN vs GN	PyTorch	Penn-Fudan	CV engineering

19. Quick Revision

Key idea: normalize over chosen axes. Formula: $(x - \text{mean}) / \sqrt{\text{var} + \text{eps}}$. Use by architecture. Trap: BN with tiny batch. Interview one-liner: normalization variants differ mainly by which tensor dimensions define the statistics.

20. Final Cheat Sheet

Definition: activation/stat normalization family. Input/output: tensor to normalized tensor. Steps: choose axes, compute stats, normalize, affine. Hyperparameters: eps, groups, momentum. Metrics: stability/accuracy. Pros: faster training. Cons: axis/batch sensitivity. Best use: architecture-specific stabilization.

Mixed Precision Training

1. Overview

Mixed precision training uses lower precision formats such as FP16 or BF16 for faster computation and lower memory, while keeping selected operations in FP32 for stability.

2. Intuition

Use cheap smaller numbers where safe, and full precision where accuracy or stability needs it.

3. Prerequisites

Floating-point formats, GPU training, PyTorch training loop, gradients, numerical overflow/underflow.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
FP16/BF16	Lower precision types	Faster tensor cores	NVIDIA GPUs	Range vs precision
Autocast	Chooses op precision	Easy mixed precision	<code>torch.autocast</code>	PyTorch API
GradScaler	Prevents FP16 underflow	Stable gradients	loss scaling	Why needed
Master weights	FP32 parameters/state	Stable updates	optimizer state	Training stability

5. Algorithm / Working Process

Run forward under autocast, compute loss, scale loss if using FP16, backpropagate scaled gradients, unscale/check for overflow, optimizer step, update scale.

6. Mathematical Foundation

Loss scaling:

```
L_scaled = S * L
grad_scaled = S * grad
grad = grad_scaled / S
```

Scaling avoids tiny gradients underflowing to zero in FP16.

7. Practical Implementation

```
import torch
import torch.nn as nn
```

```
model = nn.Linear(100, 10).cuda()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
scaler = torch.cuda.amp.GradScaler()
criterion = nn.CrossEntropyLoss()

x = torch.randn(32, 100, device="cuda")
y = torch.randint(0, 10, (32,), device="cuda")

optimizer.zero_grad()
with torch.cuda.amp.autocast():
    logits = model(x)
    loss = criterion(logits, y)
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

8. Code Explanation

Autocast runs safe operations in lower precision. GradScaler protects gradients from underflow and skips updates if overflow is detected.

9. Training / Evaluation

Use AMP on supported GPUs. BF16 often needs no gradient scaling due to wider exponent range. Validate that metrics match FP32 baseline.

10. Complexity and Cost

Reduces activation memory and speeds matrix multiplications. Optimizer state may still be FP32, so total memory savings depend on setup.

11. Common Use Cases

CNN/Transformer training, LLM fine-tuning, diffusion models, large batch training.

12. Common Mistakes

Using AMP on CPU-only training, forgetting scaler for FP16, not checking numerical parity, forcing unstable ops into FP16, assuming all GPUs accelerate BF16.

13. Edge Cases / Limitations

Some operations are numerically sensitive. Very small gradients can underflow. Hardware support varies.

14. Variations

FP16 AMP, BF16 AMP, FP8 training, quantized training, mixed precision inference.

15. Related Topics

Gradient scaling relates to exploding/underflow issues. Optimizers may keep FP32 states. LLM training often uses BF16.

16. Interview Questions

1. What is mixed precision?
Training with multiple numeric precisions.
2. Why use it?
Speed and memory savings.
3. FP16 issue?
Limited range causing under/overflow.
4. What is loss scaling?
Multiplying loss to preserve small gradients.
5. What is autocast?
Automatic precision selection per op.
6. BF16 vs FP16?
BF16 has wider range but fewer mantissa bits.
7. Does AMP change model quality?
It should not significantly if stable.
8. Which hardware helps?
GPUs with tensor cores.
9. Inference use?
Yes, lower precision inference is common.
10. What ops need FP32?
Often reductions/normalization/softmax-sensitive ops.

17. Practice Tasks

Train FP32 vs AMP, measure memory, measure speed, test BF16 if available, catch overflow events.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
AMP Benchmark	Speed/memory comparison	PyTorch	CIFAR/MNIST	Systems skill
BF16 Fine-Tune	Efficient text classifier	Hugging Face	IMDb	LLM relevance
Precision Debugger	Detects NaNs/overflow	PyTorch	Any	Production ML

19. Quick Revision

Key idea: faster low precision with stable high precision where needed. Formula: loss scaling. Use GPU training. Trap: FP16 underflow. Interview one-liner: mixed precision speeds training by using lower precision arithmetic without giving up FP32 stability.

20. Final Cheat Sheet

Definition: multi-precision training. Input/output: same model, lower memory compute. Steps: autocast, scale, backward, step. Hyperparameters: dtype, scale. Metrics: throughput, memory, validation parity. Pros: speed/memory. Cons: numerical issues. Best use: modern GPU deep learning.

Neural Tangent Kernel

1. Overview

The Neural Tangent Kernel (NTK) describes how infinitely wide neural networks behave during training. It connects deep learning with kernel methods and helps researchers analyze optimization and generalization.

2. Intuition

In very wide networks, parameter updates become tiny relative to width, so the network behaves like a kernel machine around its initialization.

3. Prerequisites

Kernel methods, gradients, Jacobians, infinite-width limits, basic functional analysis intuition, neural network training.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Kernel	Similarity function	Defines function class	RBF kernel	Link to SVM/Gaussian process
Tangent features	Gradients w.r.t. parameters	Linearized model features	<code>grad_theta f(x)</code>	NTK definition
Infinite width	Width tends to infinity	Kernel becomes fixed	wide MLP	Theory assumption
Lazy training	Function changes linearly	Simplifies analysis	small parameter movement	Research question

5. Algorithm / Working Process

At initialization, compute gradients of model output with respect to parameters for two inputs. Their inner product defines NTK similarity. In infinite width, training dynamics follow kernel gradient descent.

6. Mathematical Foundation

For network `f(x; theta)`:

$$K_NTK(x, x') = grad_theta\ f(x; theta)^T\ grad_theta\ f(x'; theta)$$

Linearized model:

$$f(x; \theta) \approx f(x; \theta_0) + \text{grad}_{\theta} f(x; \theta_0)^T (\theta - \theta_0)$$

7. Practical Implementation

```
import torch
import torch.nn as nn

model = nn.Sequential(nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 1))

def grad_features(x):
    model.zero_grad()
    y = model(x.unsqueeze(0)).squeeze()
    grads = torch.autograd.grad(y, model.parameters(), retain_graph=True)
    return torch.cat([g.flatten() for g in grads])

x1 = torch.tensor([1.0, 0.0])
x2 = torch.tensor([0.5, 0.5])
ntk_value = torch.dot(grad_features(x1), grad_features(x2))
print(ntk_value.item())
```

8. Code Explanation

The code computes gradient features for two inputs and takes their dot product, which is the finite-network empirical NTK value.

9. Training / Evaluation

NTK is mainly a research analysis tool, not a standard production training method. It helps reason about why very wide networks can fit data.

10. Complexity and Cost

Computing full NTK for n samples requires an $n \times n$ kernel and expensive parameter-gradient features, often impractical for large datasets.

11. Common Use Cases

Theory of deep learning, infinite-width analysis, kernel approximations, studying trainability, comparing architectures.

12. Common Mistakes

Assuming NTK fully explains practical finite networks, ignoring feature learning, confusing NTK with ordinary activation kernels, treating it as an optimizer.

13. Edge Cases / Limitations

Real networks often learn features significantly, especially at finite width. NTK can underrepresent representation learning and data-dependent adaptation.

14. Variations

Empirical NTK, convolutional NTK, infinite-width NTK, neural network Gaussian process, mean-field regime.

15. Related Topics

Kernel methods, Gaussian processes, weight initialization, lottery ticket hypothesis, overparameterization theory.

16. Interview Questions

1. What is NTK?
Kernel from parameter-gradient inner products.
2. Formula?
 $\text{grad_theta } f(x)^T \text{ grad_theta } f(x')$.
3. Why important?
Analyzes infinite-width training.
4. Is it practical for production?
Mostly research/theory.
5. What is lazy training?
Function evolves near initialization.
6. NTK vs feature learning?
NTK assumes limited feature movement.
7. What is empirical NTK?
NTK computed for finite network.
8. Relation to kernels?
Defines similarity and kernel regression dynamics.
9. Why infinite width?
Kernel becomes deterministic/fixed.
10. Limitation?
Does not fully explain finite deep feature learning.

17. Practice Tasks

Compute empirical NTK for tiny MLP, compare wide vs narrow networks, solve kernel regression with NTK matrix, read an NTK paper summary, visualize kernel matrix.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Empirical NTK Explorer	Computes NTK matrices	PyTorch	Synthetic 2D	Research depth

Project	What it does	Tech stack	Dataset	Resume value
Wide Network Study	Tests lazy behavior	PyTorch	MNIST subset	Theory experiment
Kernel vs MLP	Compares predictions	NumPy/PyTorch	Toy regression	ML theory bridge

19. Quick Revision

Key idea: infinite-width networks behave like kernel methods. Formula: gradient-feature dot product. Use for theory. Trap: overclaiming practical explanation. Interview one-liner: NTK is the kernel induced by neural network parameter gradients at initialization.

20. Final Cheat Sheet

Definition: neural gradient kernel. Input/output: pair of inputs to similarity. Steps: compute parameter gradients, dot product, train as kernel in limit. Hyperparameters: architecture/init. Metrics: theory fit/generalization. Pros: rigorous analysis. Cons: limited practical finite-network explanation. Best use: research.

Lottery Ticket Hypothesis

1. Overview

The Lottery Ticket Hypothesis says large randomly initialized networks contain smaller subnetworks that, when trained from their original initialization, can match the full network's performance.

2. Intuition

A big network is like many possible smaller networks hidden inside it. Training and pruning can reveal a "winning ticket" that had the right initial weights and connectivity.

3. Prerequisites

Neural networks, pruning, initialization, sparsity, training loops, model compression.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Winning ticket	Sparse subnetwork with original init	Matches dense model	pruned MLP	Core claim
Pruning	Remove weights/neurons	Finds sparse model	magnitude pruning	Compression
Rewinding	Reset remaining weights to early checkpoint	Improves large-scale results	iteration 500	Original init vs rewind

Subtopic	Meaning	Why it matters	Example	Interview angle
Sparsity	Fraction removed	Efficiency target	90% sparse	Hardware reality

5. Algorithm / Working Process

Train dense network, prune low-magnitude weights, reset remaining weights to original initialization or early checkpoint, retrain sparse subnetwork, repeat if iterative pruning is used.

6. Mathematical Foundation

Mask-based sparse model:

$$f(x; m \odot \theta)$$

where m is a binary mask and $\odot$ is elementwise multiplication. Sparsity:

$$\text{sparsity} = \text{pruned_weights} / \text{total_weights}$$

7. Practical Implementation

```
import torch
import torch.nn as nn
import torch.nn.utils.prune as prune

model = nn.Sequential(nn.Linear(784, 128), nn.ReLU(), nn.Linear(128, 10))

# After normal training, prune 30% of weights by magnitude.
for module in model.modules():
    if isinstance(module, nn.Linear):
        prune.l1_unstructured(module, name="weight", amount=0.3)
```

8. Code Explanation

PyTorch pruning adds a binary mask to each linear layer's weights. Low-magnitude weights are masked out, creating a sparse subnetwork.

9. Training / Evaluation

Evaluate dense baseline, pruned model, and retrained sparse model. Track accuracy, sparsity, parameter count, inference latency, and memory. Real speedup requires sparse-aware hardware/software.

10. Complexity and Cost

Finding tickets can be expensive because it requires train-prune-retrain cycles. Sparse inference saves memory but not always wall-clock time.

11. Common Use Cases

Model compression research, pruning studies, sparse training, efficient deployment exploration.

12. Common Mistakes

Assuming parameter sparsity guarantees speedup, pruning before baseline training, comparing unfair training budgets, forgetting original initialization in LTH experiments.

13. Edge Cases / Limitations

Original LTH works best on smaller settings. Large models often require weight rewinding. Sparse acceleration depends on hardware and libraries.

14. Variations

One-shot pruning, iterative magnitude pruning, structured pruning, weight rewinding, dynamic sparse training, early-bird tickets.

15. Related Topics

Weight initialization matters because winning tickets depend on initial values. Pruning connects to model compression. NTK relates through overparameterization theory.

16. Interview Questions

1. What is LTH?
Dense networks contain trainable sparse subnetworks.
2. What is a winning ticket?
Sparse subnetwork that trains well from original/rewound init.
3. How find tickets?
Train, prune, reset, retrain.
4. What is magnitude pruning?
Remove smallest-magnitude weights.
5. Why original initialization matters?
Ticket performance depends on initial values.
6. What is weight rewinding?
Reset to early training checkpoint.
7. Does sparsity always speed inference?
No, hardware/software must exploit it.
8. Structured vs unstructured pruning?
Remove blocks/channels vs individual weights.
9. Is LTH production standard?
More research than default production.
10. Why important?
Reveals overparameterization and compression behavior.

17. Practice Tasks

Train MNIST MLP, prune 20/50/90%, retrain sparse model, compare original reset vs random reinit, measure real latency.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Winning Ticket Finder	Iterative pruning experiment	PyTorch	MNIST	Research internship value
Sparse Speed Reality	Measures actual latency	PyTorch	Any MLP	Systems honesty
Pruning Dashboard	Tracks accuracy/sparsity	Streamlit, PyTorch	CIFAR subset	Compression project

19. Quick Revision

Key idea: sparse subnetworks can match dense models. Formula: $f(x; m \odot \theta)$. Use compression research. Trap: sparsity does not guarantee speed. Interview one-liner: LTH says overparameterized networks contain lucky sparse subnetworks trainable from special initial weights.

20. Final Cheat Sheet

Definition: sparse winning-subnetwork hypothesis. Input/output: dense model to sparse mask. Steps: train, prune, reset, retrain. Hyperparameters: pruning amount, schedule, rewind step. Metrics: accuracy, sparsity, latency. Pros: compression insight. Cons: expensive search and hardware limits. Best use: pruning research.